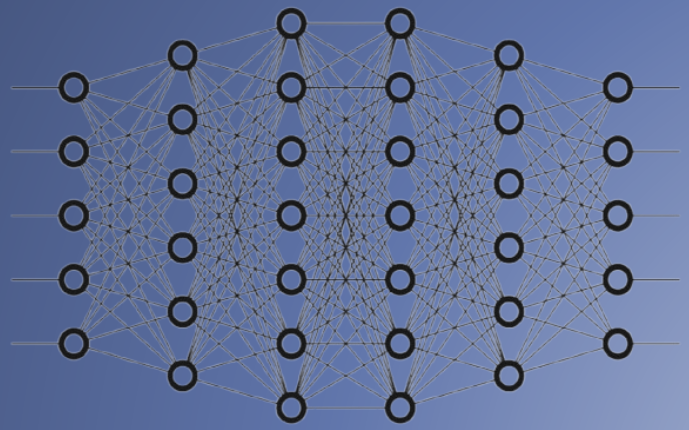
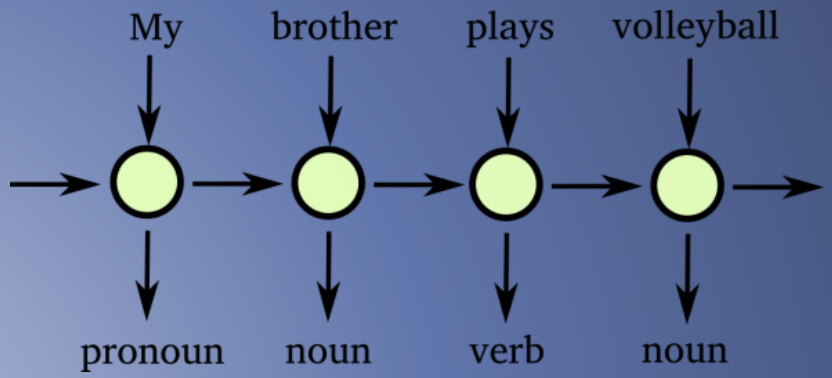
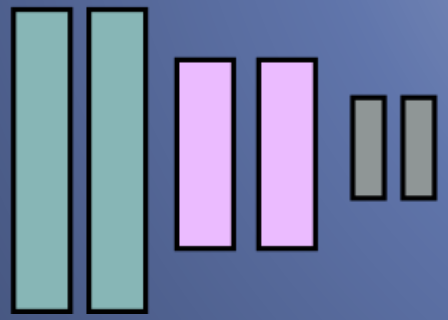
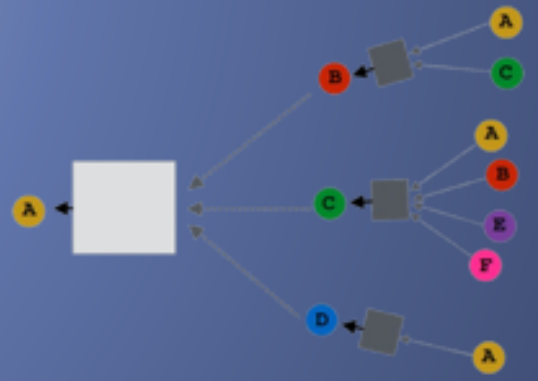




DEEP LEARNING ARCHITECTURES





AGENDA

What we are going to see

- Basic Theory
- Coding Example!

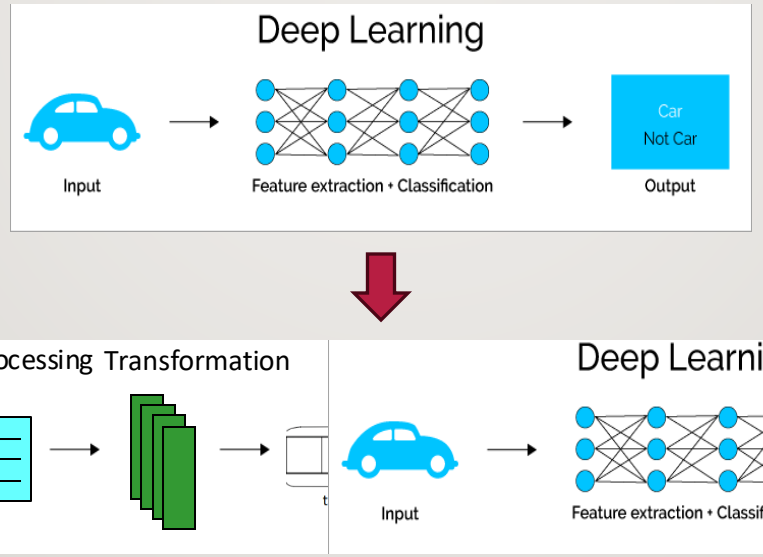
About

- Data Handling
- Deep Learning Architectures
  - Feed-Forward Neural Network (FFNN)
  - Recurrent Neural Network (RNN)
  - Graph Neural Network (GNN)
  - Convolutional Neural Network (CNN)

DEEP LEARNING PIPELINE

- Deep Learning pipeline simplify the feature extraction phase..
  - The entire process is not fully automated!

We still need to know/work on the data before giving it in input to the model!



DATA HANDLING

PRE-PROCESSING

- **Data cleaning**: **Noise** and **outliers <u>may</u>** still affect the pipeline
  - Noise less, outlier more
- **Handle redundancy**: duplicates may artificially increase the importance of certain points
  - Reducing generalization
- **Handle missing values**: points with missing values cannot be used by the neural network
  - Eliminate data points
  - Estimate missing values
  - **Ignore the features** with missing value during the analysis
  - Replace with all possible values (weighted by their probabilities)
    - If it makes sense..

What do we use? Pandas

DATA TRANSFORMATION

- Data transformation depends the type of data on hand
  - **Categorical attributes that are <u>nominal</u>**: colours, country name, encryption algorithm, internet port, words, …
  - **Categorical attributes that are <u>ordainable</u>:** educational level, cloth size, access control level, threat level, …
  - **Numerical attributes**: temperature, height, byte sent, number of packets, passwords attempts, …

DATA TRANSFORMATION: CATEGORICAL AND ORDINAL ATTRIBUTES

- **One-hot-encoding**: converts each categorical value into a **binary vector**
  - **1 input neuron for each** possible feature **value**!
  - **PROS**: frequently use as it is simple and effective for small categorical data
  - **CONS**: the number of input neurons may explode for high cardinality features
    - What about text, English has 171k+ words ➔ 171k input neurons?!?!?

ID	Encryption Algorithm (EA)
1	AES
2	RSA
3	DES
4	AES

One-hot-Encoding

ID	EA_AES	EA_RSA	EA_DES
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0

- **Label encoding**: assigns an integer to each category
  - Only **1 input neuron** for the **feature**
  - **PROS** : Simple and memory efficient
  - **CONS**: Implies ordinal relationships, which may mislead the model
    - Can be used if ordainable data, or to encode textual labels

ID	Access Control Level
1	Medium
2	High
3	Low
4	Medium

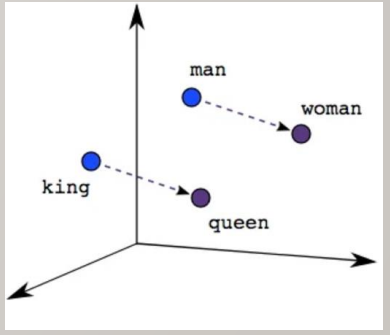
Label Encoding

ID	Encoded Access Control Level
1	2
2	3
3	1
4	2

- **Embeddings**: create a latent space, where we map each possible feature value with a **vector**
  - **1 input neuron for each coordinate** in the latent space
  - **PROS**: can represent high-cardinality fetures with a small vector
  - **PROS**: similar concepts can be represented near by each other
  - **CONS**: can be computational expensive and requires enough data for traning

ID	Word
1	King
2	Queen
3	Man
4	Woman

ID	Word Latent Space
1	[0.5,2,1]
2	[1.7,1.8,0.5]
3	[0.7,2,0.2]
4	[1.9,1.8,-0-3]



DATA TRANSFORMATION: NUMERICAL ATTRIBUTES

- 1 input neuron for each feature
- **Normalization (min-MAX scaling)**: scales the value using minimum and maximum values of the feature
  - **PROS**: keeps data within a fixed range
  - **CONS**: sensitive to outliers
- **Standardization (Z-score scaling):** scales the value using mean and variance
  - **PROS:** works also in presence of with outliers
  - **CONS:** there is no specific boundaries

What do we use? Sklearn

$$z = \frac{(x - x_{min})}{(x_{MAX} - x_{\min})}$$

x = feature

x_{MAX} = max value of the feature

x_{min} = min value of the feature

$$z = \frac{(x - \mu)}{\sigma}$$

x = feature

σ = variance

μ=mean

SPLITTING DATA

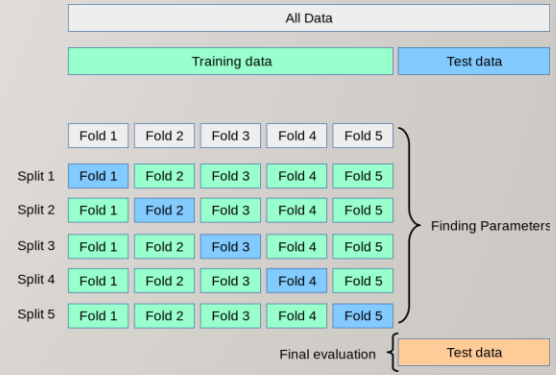
- Understand how to correctly split the data if **fundamental** to have reliable performance estimation and generalize the model!
- If possible, data should be split in
  - **Train**: To train the model
  - **Validation**: To chose the best model hyperparameters based on performance metrics
  - **Testing (if possible)**: To assess if the model is general
    - I.e., Performance with never seen data is similar to validation performance
- Different validation techniques are available
- Validation problems to address
  - Class distribution
  - Cost of misclassification
  - Dataset size
  - Data leakage

SPLITTING DATA: VALIDATION TECHNIQUES

Divide the dataset in multiple parts

- **Stratified sampling** to generate partitions
  without replacement
- **Bootstrap**
  - Sampling with replacement
- **Hold-out**: Fixed partitioning
  - E.g., reserve 70% for training and 20% for validation, 10% for testing
  - Appropriate for large datasets
    - May be repeated several times
- **Cross validation**: partition data into k disjoint subsets (i.e., folds)
  - k-fold: train on k-1 partitions, test on the remaining one
    - Repeat for all folds
  - Reliable accuracy estimation, not appropriate for very large datasets/small datasets
- **Leave-one-out**: is a cross validation for k=n
  - Only appropriate for very small datasets

What do we use? Sklearn



Schikit-learn: Cross-validation

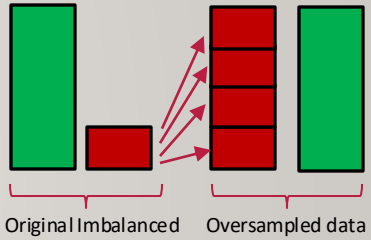
SPLITTING DATA PROBLEMS: CLASS DISTRIBUTION

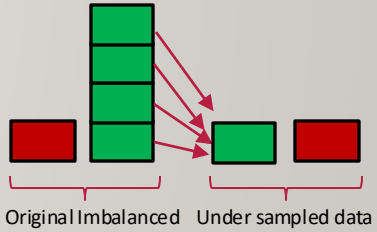
Some problem may inherit very **unbalanced** classes, e.g., a binary classifier may have the **majority class** with 99% of the samples

- Can create bias during model training, or the model being able to describe well only the majority class(es)

Data-Level Techniques (Resampling): Act on the the input data

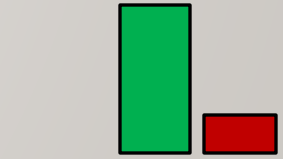
- **Oversampling the Minority Class**
  - **Duplicate** or generate **synthetic samples** for the minority class
    - SMOTE (Synthetic Minority Over-sampling Technique): Creates synthetic samples by interpolating between existing minority samples
    - ADASYN (Adaptive Synthetic Sampling): A variant of SMOTE that generates more synthetic samples for difficult-to-learn instances
- **Under sampling the Majority Class**
  - **Randomly** remove samples from the majority class
  - **Cluster-based**: uses clustering to select representative samples instead of random deletion





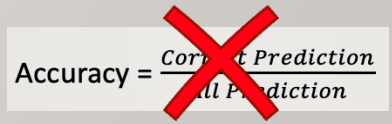
Algorithm-Level Techniques: Adjust the learning process to give more importance to minority classes.

- **Class Weights Adjustment**: Assign higher loss weights to the minority class and lower weights to the majority class
- **Focal Loss**: Modify the loss function to focus on more difficult-to-classify samples with class weight and focus factor



Choose the right metric: some performance metrics may be misleading

- If majority class is 99%, predicting the **always it** will result in **99% Accuracy!**
- Choose per-class metrics or use weighted metrics (micro avg instead of macro)



SPLITTING DATA PROBLEMS PROBLEMS: COST OF MISCLASSIFICATION

Errors in some classes may be more relevant than others

- E.g., flag a software as potential malware is **less harmful** that miss a malware
- **Use the right classification metric:** Eventually focusing more on the most relevant class(es)
- **Class Weights Adjustment**: Assign higher loss weights to the minority class and lower weights to the majority class
- **Focal Loss**: Modify the loss function to focus on more difficult-to-classify samples

SPLITTING DATA PROBLEMS PROBLEMS: DATASET SIZE

A dataset may be too small for partitioning it with a hold-out technique

- Use cross-validation of leave-one-out technique
- Data augmentation to generate **synthetic samples**
  - **SMOTE** (Synthetic Minority Over-sampling Technique): Creates synthetic samples by interpolating between existing minority samples
  - **ADASYN** (Adaptive Synthetic Sampling): A variant of SMOTE that generates more synthetic samples for difficult-to-learn instances

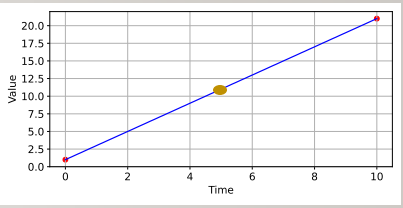
Do not face the problem end-to-end but learn a representation first

- Use a pre-trained model and fine-tune it
- Use a semi-supervised approach if unlabelled data is available

SPLITTING DATA PROBLEMS PROBLEMS: DATA LEAKAGE

When information in the training data influence the validation process

- Toy case with sequential points having **temporal correlation**
  - **T**wo red dots in the training set at time t-1 (0) and t+1 (10) second
- What is the value at **time t (5)** in the validation set?
  - Easy to predict! In real life to predict t, do we **always** have t+1?? **No!**



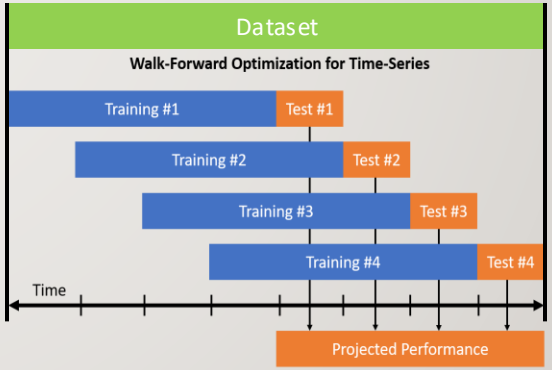
When data contains feature **biasing** the model

- Suggesting too specific description, that may compromise model generality!
- Denial-of-Service (DDoS) attack (many request) always seen only on TCP port 80
  - If we give to the model the port, it may learn that DoS behaviour is related to port 80!

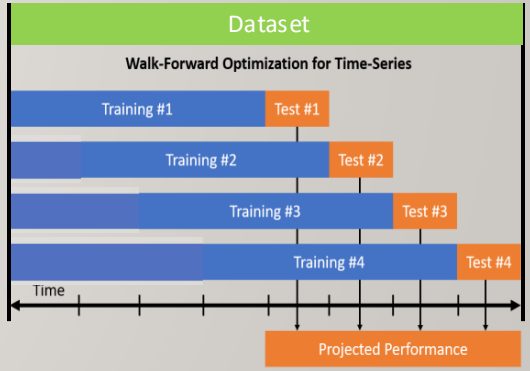
Study your problem!

- If there are temporal patterns use time-based splitting
- Time is not the only possible problem!
  - Toy case with an image where the image is composed by 3 chunks. c_0 and c_2 in the training set – c_1 is in the validation set
    - Can you predict what is c_1? **Too easy**!!
- **Bias**: study if there are identifiers or pseudo-identifiers or bias in your data and do not use them for training the model

Sliding window



Expanding window

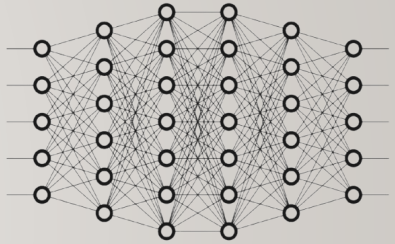


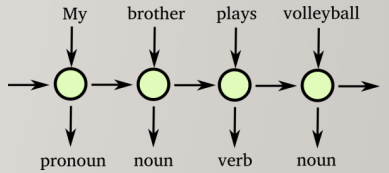
C_0 C_1 C_2

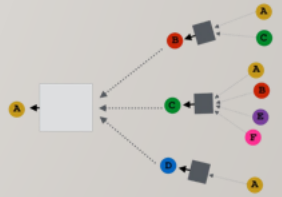
DEEP LEARNING ARCHITECTURES

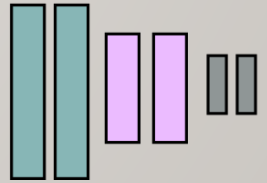
DEEP LEARNING ARCHITECTURES

- We will study different architecture
  - Each is designed to address specific tasks
- Feed-Forward Neural Networks (**FFNN**): born to model numerical data
  - E.g., Classify a malware based on predefined features
  - Can be used alone or after other Neural Networks
    - E.g., In image classification, first the image is encoded in numerical form than FFNN classify it
- Recurrent Neural Network (**RNN**): born to model sequential data
  - RNN is a chain of FFNN that remembers the history of previous data points
    - E.g., to predict next time-series values of the volume of traffic, or next word in a sentence
- Graph Neural Network (**GNN**): born to model graphs
  - GNN use the connections between nodes by aggregating and transforming neighbourhood information
    - E.g., malware detection, social network analysis, fraud detection
- Convolutional Neural Network (**CNN**): born to model images
  - CNN use specific **convolutional filters** to extract from images features that describe them. They have been used also in other fields including cybersecurity
    - E.g., encoding the Malware as an image and use a CNN to extract features and a FFNN to classify it







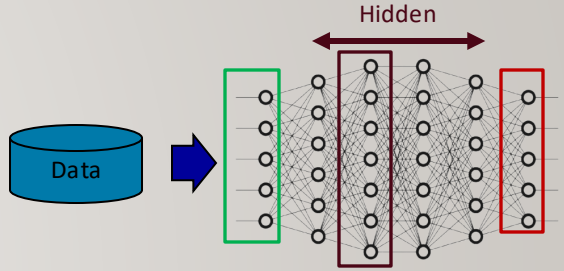


FEED-FORWARD NEURAL NETWORK

FEED-FORWARD NEURAL NETWORK: EVERYTHING STARTS FROM HERE

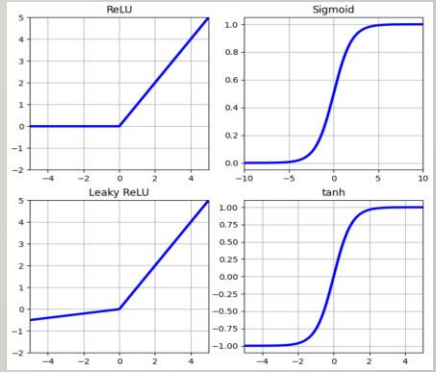
How to **preprocess** and **transform** the data:

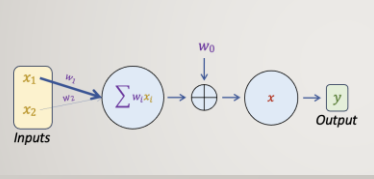
- Data must be **numbers** ➔ Non numerical features must be be encoded!
  - E.g., categorical, the label, the dates, time, etc.
  - **NOTE**: Features may be numbers (e.g., port) but still **categorical**!!
- **Numerical**: Normalization/standardization helps in improving the performance



The **architecture**

1. **Input layer**: how many nodes?
   - Based on preprocessing and transformation
2. **Hidden-layers**: How many?
   - Start with a few then increase if required
   - How many nodes **per** layer?
     - **Common practice**: use power of 2 for computational efficiency
   - Which activation function **per layer**?
     - Depending on how deep is the network, e.g., ReLu, Leaky ReLU.. Sigmoid, tanh?
3. **Output layer**: how many?
   - Based on the number of classes
   - Which activation function?
     - Depending on the problem/Loss function, e.g., Sigmoid, SoftMax, ..?
4. **Weights** initialization: automatic?
   - Typically automatic: **set the random seeds** for **reproducibility** ➔ otherwise different run will have different performance
   - Set the weights manually to target **specific network aspects** during the experiments

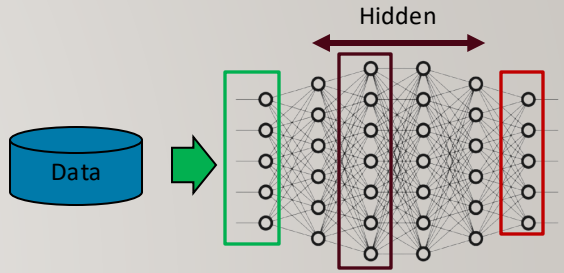




FEED-FORWARD NEURAL NETWORK: EVERYTHING STARTS FROM HERE

How to optimize the architecture

5. How to input the data?
   - All data together or mini batches?
6. Which Loss Function?
   - If unbalanced, should we consider weights?
7. Which Optimizer?
   - How many epochs and what learning rate?



In case of overfitting: How to modify the Neural Network Architecture?

7. Modify the weights computation, modify the class weights, add drop-out layers
   - If strong overfitting.. Restart the game from the data preprocessing/design the architecture

FEED-FORWARD NEURAL NETWORK: A SUMMARY OF HYPERPARAMETERS AND BEST PRACTICES

Hyperparameter	Best Practices
# Layers	Start with 3-5 hidden layers, increase if needed
# Neurons per Layer	Use powers of 2 (64, 128, 256), avoid too many
Activation	ReLU, LeakyReLU, etc. for hidden layers, Softmax/Sigmoid for output
Weight Initialization	PyTorch automatically initialize the weights with different methods based on the activation function in the layer. Manual initialization is possible
Batch Size	32-256, tune experimentally
Loss Function	Choose based on the task
Optimizer	Adam (Default), SGD+Momentum for generalization
Learning Rate	Start with 0.001, use LR scheduler.
Epochs & Early Stopping	Monitor validation loss, stop if overfitting. The number of epochs can drastically change based on the optimizer and Learning Rate!
Regularization	Dropout (0.2-0.5) + L2 (0.01)

LOADING THE DATA: FROM RAW DATA TO TENSOR

After preparing the data.. Machine Learning **starts** from loading the data in pyTorch

- Data data must be **numbers** that are then converted into Tensors
- A tensor is a multi-dimensional matrix containing elements that **are numbers** of a **single data type**
  - Multi-dimensional means also 1 dimension, i.e., a number
  - Optionally, **we can specify the data type**
    - Useful when there are mismatch between the desired data type and tensor conversion
      - E.g., the label should be *torch.long* not *torch.float32*

PyTorch

Method	Use Case
torch.tensor()	General conversion from Python lists, tuples, matrix, etc.
torch.zeros(), torch.ones(), torch.arange()	Generate specific tensors Zeros is **useful to create masks of other data**.. With matrix mutiplication
torch.rand(), torch.randn(), torch.randint()	Create tensors with random values

```
X_train = [[1,0.5,4],[7,6,4],[9,6,2]]
y_train = [1,0,1]

X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.long)

print(X_train_tensor)
print(y_train_tensor)
```

```
tensor([[1.0000, 0.5000, 4.0000],
        [7.0000, 6.0000, 4.0000],
        [9.0000, 6.0000, 2.0000]])
tensor([1, 0, 1])
```

```
torch.zeros(2,3)
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

```
torch.randn(2,3)
```

```
tensor([[ 0.6576,  1.0131,  0.3465],
        [ 1.2991,  0.1129, -0.2319]])
```

LOADING THE DATA: USING MINI BATCHES

Hyperparameter	
# Layers	
# Neurons per Layer	
Activation	
Weight Initialization	
Batch Size	
Loss Function	
Optimizer	
Learning Rate	
Epochs & Early Stopping	
Regularization	

This is the Input data for the ML task

- We can combine multiple tensors using *TensorDataset()*
  - To join features and labels

1. Use the TensorDataset() ➔ without using mini-batches
   - **WARNING**: May not fit in memory!!
2. Use the the TensorDataset() and mini-batches with the *DataLoader()*
   - We specify the dataset, the batch size, and if shuffle the data or not
     - Shuffle determines whether the dataset is randomly shuffled at the beginning of each epoch
       - The data is given in input to the model in a different order than in the dataset
       - **PRO**: Improves generalization by helping the model learn more robust features reducing overfitting
       - **WARNING**: In sequential data the order is **important**! If they are defined with different data points, i.e., not on a single row and we want to evaluate these points as a sequence we cannot shuffle the sequence. Difference sequences **can** be shuffled

     Q: During validation/test shuffling makes sense?
       - NO! The prediction will not change
       - Randomness reduces the Loss readability and results comparison

```
from torch.utils.data import DataLoader, TensorDataset
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_dataset
```

```
<torch.utils.data.dataset.TensorDataset at 0x7f8d18545df0>
```

```
train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=True)
train_dataloader
```

```
<torch.utils.data.dataloader.DataLoader at 0x7f8d183af3d0>
```

CRATE THE FIRST NEURAL NETWORK ARCHITECTURE: WITH 1 LINEAR LAYERS

Hyperparameter	
# Layers	✅
# Neurons per Layer	✅
Activation	
Weight Initialization	✅
Batch Size	
Loss Function	
Optimizer	
Learning Rate	
Epochs & Early Stopping	
Regularization	

PyTorch creates NN defining it with a class

1. Create the class to describe the NN architecture
2. The ***__init__*** function can have multiple parameters
   - E.g., number of input neurons, number of layers, number of neurons per layer, activation function, etc..
   - The ***super*** constructor specify that it is a NN
   - **1 hidden-layer** with 32 neurons and a number of input = to the number of input neurons
   - **The second layer** has the input size as the size of the previous layer. In this case this is this the **output layer**
     - The neurons in the output layer depends on: the task regression/classifier, number of classes, Loss function
   - The output of the last layer is called ***logit***
     - Logits are the **raw, unnormalized** outputs of a neural network **before** applying an activation function. The range is $[-\infty, \infty]$
3. The ***forward*** function defines how data pass trough the NN

```
import torch.nn as nn

# Define a neural network class for multiclass classification that inherits from nn.Module
class MulticlassLinearNN(nn.Module):

    # The constructor initializes the network's layers input_size is the number of input neuorons
    def __init__(self, input_size):
        # Call the constructor of the parent class (nn.Module) the neural network
        super(MulticlassLinearNN, self).__init__()

        # Define the first hidden (linear) layer
        self.hidden_layer = nn.Linear(in_features=input_size, out_features= 32)

        # Define the output layer. Based on the problem/loss the number output neuron changes
        self.output_layer = nn.Linear(in_features=32, out_features= 3)

    # The forward method defines how data flows through the network
    def forward(self, x):
        # Pass input 'x' through the first fully-connected layer
        x = self.hidden_layer(x)
        # Pass the output of the first hidden-layer through the output fully-connected layer
        x = self.output_layer(x)

        # Return the final output; note that no activation is applied here!
        # This ouput is called *logit*
        return x
```

```
n_input_nodes = 10
model = MulticlassLinearNN(input_size=n_input_nodes)
model
```

```
MulticlassLinearNN(
  (hidden_layer): Linear(in_features=10, out_features=32, bias=True)
  (output_layer): Linear(in_features=32, out_features=2, bias=True)
)
```

Q: Is there **any** non-linearity here? **A**: No, there is not a non-linear activation function

CREATE THE FIRST NEURAL NETWORK ARCHITECTURE: ADD NONLINEARITY

Hyperparameter	
# Layers	✅
# Neurons per Layer	✅
Activation	✅
Weight Initialization	✅
Batch Size	
Loss Function	
Optimizer	
Learning Rate	
Epochs & Early Stopping	
Regularization	

- To add nonlinearity, we can specify the activation function
  - Different layers can use different activation function!
- How to apply the activation function is specified in the *forward* function

Q: On the last layer, should we **always** apply an activation function?

A: Yes to map from logits to probabilities

- E.g., For a multiclass problem we should apply the ***softmax*** to map logits to probabilities..

… Why **here** we do not not specify it??

```python
import torch.nn as nn

# Define a neural network class for multiclass classification that inherits from nn.Module
class MulticlassNoNLinearNN(nn.Module):
    ... architecture details
        # Specify the activation function to introduce non linearity
        self.activation_function = nn.ReLU()

    # The forward method defines how data flows through the network
    def forward(self, x):
        # Pass input 'x' through the first fully-connected layer - and use the non lineary output function
        x = self.activation_function(self.hidden_layer(x))
        # Pass the output of the first hidden-layer through the output fully-connected layer
        x = self.output_layer(x)

        # Return the final output; note that no activation is applied here!
        # This ouput is called *logit*
        return x
```

```python
n_input_nodes = 10
model = MulticlassNoNLinearNN(input_size=n_input_nodes)
model
```

```
MulticlassNoNLinearNN(
  (hidden_layer): Linear(in_features=10, out_features=32, bias=True)
  (output_layer): Linear(in_features=32, out_features=3, bias=True)
  (activation_function): ReLU()
)
```

```python
# Example "sparse" logits for a batch of 2 samples and 3 classes.
# Here, one value in each sample is much higher than the others.
logits = torch.tensor([
    [-10.0, 20.0, -15.0],  # For sample 1, the second class is dominant
    [0.1, -20.0, 25.0]      # For sample 2, the third class is dominant
])

# Apply softmax along the class dimension (dim=1) to convert logits into probabilities
probabilities = F.softmax(logits, dim=1)

# Extract the predicted class for each sample by taking the index of the maximum probability
predicted_classes = torch.argmax(probabilities, dim=1)

print("Sparse Logits:\n", logits)
print("Softmax Probabilities:\n", probabilities)
print("Predicted Classes:", predicted_classes)
```

```
Sparse Logits:
 tensor([[-10.0000,  20.0000, -15.0000],
        [  0.1000, -20.0000,  25.0000]])
Softmax Probabilities:
 tensor([[9.3576e-14, 1.0000e+00, 6.3051e-16],
        [1.5349e-11, 2.8625e-20, 1.0000e+00]])
Predicted Classes: tensor([1, 2])
```

SPECIFY HOW TO TRAIN THE NETWORK: LOSS & OPTIMIZER

Hyperparameter	
# Layers	✅
# Neurons per Layer	✅
Activation	✅
Weight Initialization	✅
Batch Size	
Loss Function	✅
Optimizer	✅
Learning Rate	✅
Epochs & Early Stopping	
Regularization	

- The criterion specify the **Loss function**
  - Multiclass ➔ CrossEntropyLoss()
    - How does it work in PyTorch??
      - "The ***input*** **is expected to contain the unnormalized logits** for each class (which do *not* need to be positive or sum to 1, in general)" ➔ We must not specify the softmax!
    - **REMEMBER**: Before using a Loss check the documentation
    - We can specify ***weights*** to balance the loss of the classes and other parameters..
- Finally, we can specify the **optimizer** to use on the NN architecture and its details
  - Learning rate, L2 weights normalization, momentum, etc..

```python
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
model = MulticlassNoNLinearNN(input_size=n_input_nodes)
optimizer = optim.AdamW(model.parameters(), lr=0.01)
print(model)
print(criterion)
print(optimizer)
```

```
MulticlassNoNLinearNN(
  (hidden_layer): Linear(in_features=10, out_features=32, bias=True)
  (output_layer): Linear(in_features=32, out_features=3, bias=True)
  (activation_function): ReLU()
)
CrossEntropyLoss()
AdamW (
Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.01
    maximize: False
    weight_decay: 0.01
)
```

TRAIN AND VALIDATE THE NETWORK: THE TRAINING LOOP **NO MINI-BATCHES**

Hyperparameter	
# Layers	✅
# Neurons per Layer	✅
Activation	✅
Weight Initialization	✅
Batch Size	
Loss Function	✅
Optimizer	✅
Learning Rate	✅
Epochs & Early Stopping	✅
Regularization	

- Specify how long to train: **num_epocs**
- Track the training and validation losses
  - TO study when (and if) the model converges, overfitting/underfitting

1. Set the model to training mode
2. At each epoch restart the gradient computation
3. Train the model
4. Compute the loss
5. Backpropagate it
6. Optimize it
7. Save the loss of training set during that epoch

```python
# 4. Train the network without mini-batches (using the full dataset each epoch).
num_epochs = 50                 # Total number of epochs for training
train_losses = []               # List to store training loss per epoch
val_losses = []                 # List to store validation loss per epoch

for epoch in range(num_epochs):
    # ----- Training Phase -----
    model.train()               # Set the model to training mode (affects layers like dropout)
    optimizer.zero_grad()       # Zero out the gradients for the optimizer
    outputs = model(X_train)    # Perform forward pass on the full training dataset
    loss = criterion(outputs, y_train)  # Compute the loss using the full training set
    loss.backward()             # Backpropagate to compute gradients
    optimizer.step()            # Update model parameters based on gradients

    train_loss = loss.item()    # Extract the scalar loss value
    train_losses.append(train_loss)  # Append training loss for this epoch
```

TRAIN AND **VALIDATE** THE NETWORK: THE TRAINING LOOP **NO MINI-BATCHES**

Hyperparameter	
# Layers	✅
# Neurons per Layer	✅
Activation	✅
Weight Initialization	✅
Batch Size	
Loss Function	✅
Optimizer	✅
Learning Rate	✅
Epochs & Early Stopping	✅
Regularization	

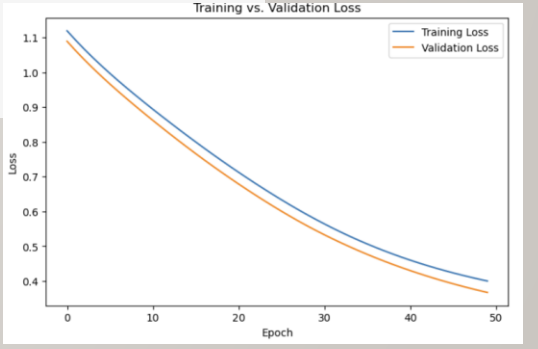
6. If validation set is available
   1. Set the model in validation model
   2. set that the gradient does not have to be computed
   3. Perform the validation
   4. Compute and track the loss
7. After each epoch (or periodically) print the loss value
8. After all epochs plot the trend of the Loss Function

Q: Is this a good Loss value?

A: Not really… because the curves did not converge yet

```python
# 4. Train the network without mini-batches (using the full dataset each epoch).
num_epochs = 50                 # Total number of epochs for training
train_losses = []               # List to store training loss per epoch
val_losses = []                 # List to store validation loss per epoch

for epoch in range(num_epochs):
    # ----- Training Phase -----
    model.train()               # Set the model to training mode (affects layers like dropout)
    optimizer.zero_grad()       # Zero out the gradients for the optimizer
    outputs = model(X_train)    # Perform forward pass on the full training dataset
    loss = criterion(outputs, y_train)  # Compute the loss using the full training set
    loss.backward()             # Backpropagate to compute gradients
    optimizer.step()            # Update model parameters based on gradients

    train_loss = loss.item()    # Extract the scalar loss value
    train_losses.append(train_loss)  # Append training loss for this epoch

    # ----- Validation Phase -----
    model.eval()                # Set the model to evaluation mode (disables dropout, etc.)
    with torch.no_grad():       # Disable gradient computation for validation
        val_outputs = model(X_val)    # Forward pass on the validation dataset
        val_loss = criterion(val_outputs, y_val)  # Compute validation loss
        val_losses.append(val_loss.item())  # Append validation loss for this epoch

    # Print loss information for each epoch
    print(f"Epoch [{epoch+1}/{num_epochs}] Train Loss: {train_loss:.4f} | Val Loss: {val_loss.item():.4f}")
```

```python
# 5. Plot training and validation loss curves.
plt.figure(figsize=(8, 5))                  # Create a new figure with specific size
plt.plot(train_losses, label='Training Loss')  # Plot training loss over epochs
plt.plot(val_losses, label='Validation Loss')  # Plot validation loss over epochs
plt.xlabel("Epoch")                         # X-axis label
plt.ylabel("Loss")                          # Y-axis label
plt.title("Training vs. Validation Loss") # Plot title
plt.legend()                                # Display legend
plt.show()                                  # Show the plot
```



TRAIN AND VALIDATE THE NETWORK: THE TRAINING LOOP **WITH MINI-BATCHES**

Hyperparameter	
# Layers	✅
# Neurons per Layer	✅
Activation	✅
Weight Initialization	✅
Batch Size	✅
Loss Function	✅
Optimizer	✅
Learning Rate	✅
Epochs & Early Stopping	✅
Regularization	

1. To use mini-batches use the data loader to specify the batch size and if shuffle
2. Modify the training loop to include iterations over the min-batches
3. Compute the epoch loss as the average over all mini-batches
   - **NOTICE**: we compute the weighted average loss over the epochs
     - The weights of the average consider the mini-batch size
     - The average loss consider the dataset size

```python
# Create DataLoader
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

```python
# Define training parameters
num_epochs = 50

train_losses = []
val_losses = []

# Training loop
for epoch in range(num_epochs):  # Train for epochs
    train_loss = 0
    val_loss = 0
    model.train()  # Set model to training mode
    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()  # Clear previous gradients
        outputs = model(batch_X)  # Forward pass
        loss = criterion(outputs, batch_y)  # Compute loss
        loss.backward()  # Backpropagation
        optimizer.step()  # Update model parameters
        train_loss += loss.item() * batch_X.size(0)
    train_loss /=len(train_dataset)
    train_losses.append(train_loss)  # Store training loss

    #To compute Validation loss during training
    model.eval()  # Set model to evaluation mode
    with torch.no_grad():  # Disable gradient computation for validation
        for batch_X, batch_y in val_loader:
            val_outputs = model(batch_X)  # Forward pass
            loss = criterion(val_outputs, batch_y)  # Compute validation loss
            val_loss += loss.item() * batch_X.size(0)
        val_loss /=len(val_dataset)
        val_losses.append(val_loss)  # Store validation loss
    print(f'Epoch {epoch+1}/{num_epochs}, Train Loss: {train_losses[-1]:.4f}, Val Loss: {val_losses[-1]:.4f}')
```

```
Epoch 1/50, Train Loss: 0.3536, Val Loss: 0.3153
```

IMPROVE OVERFITTING WITH REGULARIZATION

Hyperparameter	
# Layers	✅
# Neurons per Layer	✅
Activation	✅
Weight Initialization	✅
Batch Size	✅
Loss Function	✅
Optimizer	✅
Learning Rate	✅
Epochs & Early Stopping	✅
Regularization	✅

1. Introduce dropout layers
   - **NOTICE**: Play with the dropout value from low (0.2) to high (0.5)
   - Does not remove the neurons or weights
   - Set the output of some nodes to **zero during training**
   - Nodes **are used during prediction** (in eval mode)
   - 1st dropout after at 2nd hidden layer
   - 2nd dropout after at the 4th hidden layer
2. Apply L2 normalization
   - Apply *weight_decay* to *Adam* or *AdamW* optimizer

```python
# Define a neural network class for multiclass classification that inherits from nn.Module
class MulticlassNoNLinearNNDropOut(nn.Module):
    # The constructor initializes the network's layers; input_size is the number of input neurons
    def __init__(self, input_size):
        # Call the constructor of the parent class (nn.Module)
        super(MulticlassNoNLinearNNDropOut, self).__init__()

        # Define the first hidden (linear) layer: maps input to 64 neurons
        self.hidden_layer1 = nn.Linear(in_features=input_size, out_features=64)

        # Define the second hidden (linear) layer: maps from 64 to 32 neurons
        self.hidden_layer2 = nn.Linear(in_features=64, out_features=128)

        # Define the third hidden (linear) layer: maps from 32 to 32 neurons
        self.hidden_layer3 = nn.Linear(in_features=128, out_features=256)

        # Define the fourth hidden (linear) layer: maps from 32 to 32 neurons
        self.hidden_layer4 = nn.Linear(in_features=256, out_features=32)

        # Define the output layer: maps from 32 neurons to 3 output classes
        self.output_layer = nn.Linear(in_features=32, out_features=3)

        # Specify the activation function to introduce non-linearity (ReLU)
        self.activation_function = nn.ReLU()

        # Define dropout layers to help prevent overfitting; p specifies the dropout probability
        self.dropout1 = nn.Dropout(p=0.5)  # First dropout layer with 50% dropout rate
        self.dropout2 = nn.Dropout(p=0.5)  # Second dropout layer with 50% dropout rate

    # The forward method defines how data flows through the network
    def forward(self, x):
        # Pass input 'x' through the first hidden layer and apply ReLU activation
        x = self.activation_function(self.hidden_layer1(x))

        # Pass through the second hidden layer, apply ReLU, then apply the first dropout layer
        x = self.activation_function(self.hidden_layer2(x))
        x = self.dropout1(x)

        # Pass through the third hidden layer and apply ReLU activation
        x = self.activation_function(self.hidden_layer3(x))

        # Pass through the fourth hidden layer, apply ReLU, then apply the second dropout layer
        x = self.activation_function(self.hidden_layer4(x))
        x = self.dropout2(x)

        # Pass the output of the last hidden layer through the output layer to obtain logits
        x = self.output_layer(x)

        # Return the final output logits (no activation applied here)
        return x
```

```python
# Define the Adam optimizer with weight decay
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.01)
```

WEIGHTS INITIALIZATION

Hyperparameter	
# Layers	✅
# Neurons per Layer	✅
Activation	✅
Weight Initialization	✂
Batch Size	✅
Loss Function	✅
Optimizer	✅
Learning Rate	✅
Epochs & Early Stopping	✅
Regularization	✅

- Custom weights initialization can be useful to run target experiments
  - **E.g., setting very small weights leads to gradient vanishing problem**

```python
import torch.nn as nn
import torch.nn.init as init  # Import initialization module

# Define a neural network class for multiclass classification that inherits from nn.Module
class MulticlassNoNLinearNNCustomW(nn.Module):

    # The constructor initializes the network's layers; input_size is the number of input neurons
    def __init__(self, input_size,w_init):
        # Call the constructor of the parent class (nn.Module)
        super(MulticlassNoNLinearNNCustomW, self).__init__()

        # Define the first hidden (linear) layer that maps input_size to 32 neurons
        self.hidden_layer = nn.Linear(in_features=input_size, out_features=32)

        # Define the output layer that maps from 32 neurons to 3 output classes
        self.output_layer = nn.Linear(in_features=32, out_features=3)

        # Specify the activation function to introduce non-linearity
        self.activation_function = nn.ReLU()

        # Initialize weights for each layer using custom initialization
        self._initialize_weights(w_init)

    # Custom method to initialize weights
    def _initialize_weights(self,w_init):
        if(w_init=="auto"):
            # Initialize the weights of the hidden_layer using Kaiming (He) initialization
            # suitable for ReLU activations
            init.kaiming_normal_(self.hidden_layer.weight, mode='fan_in', nonlinearity='relu')
            # Initialize the bias of the hidden_layer to zero.
            if self.hidden_layer.bias is not None:
                init.zeros_(self.hidden_layer.bias)

            # Initialize the weights of the output_layer using Xavier (Glorot) initialization.
            init.xavier_normal_(self.output_layer.weight)
            # Initialize the bias of the output_layer to zero.
            if self.output_layer.bias is not None:
                init.zeros_(self.output_layer.bias)

        # Custo Initialize the weights very small to force gradient vanishing
        if(w_init=="manual"):
                nn.init.normal_(self.hidden_layer.weight, mean=0.0, std=0.0001)  # Small weights for vanishing
                nn.init.normal_(self.hidden_layer.bias, mean=0.0, std=0.0001)  # Small weights for vanishing

                nn.init.normal_(self.output_layer.weight, mean=0.0, std=0.0001)  # Small weights for vanishing
                nn.init.normal_(self.output_layer.bias, mean=0.0, std=0.0001)  # Small weights for vanishing

    # The forward method defines how data flows through the network
    def forward(self, x):
        # Pass input 'x' through the first hidden layer and apply the ReLU activation function.
        x = self.activation_function(self.hidden_layer(x))
        # Pass the output of the hidden layer through the output layer (raw logits are returned)
        x = self.output_layer(x)
        # Return the final output logits (no activation applied here)
        return x
```

Auto

```
MulticlassNoNLinearNNCustomW(
  (hidden_layer): Linear(in_features=2, out_features=32, bias=True)
  (output_layer): Linear(in_features=32, out_features=3, bias=True)
  (activation_function): ReLU()
)
Hidden Layer Weights:
Parameter containing:
tensor([[ 1.6987,  0.3327],
        [-0.0805,  2.1424],
        [-0.6532, -0.8228],
        [-0.3181, -0.2400],
        [-3.4491,  0.4598],
        [ 0.2902,  0.5567],
        [-0.1260, -1.7679],
        [ 2.3409, -2.6334],
        [-1.0509, -0.9522],
        [ 0.0196, -0.6093],
        [ 1.8637, -0.4864],
        [-1.0836,  0.9947],
        [ 2.4292, -0.7943],
        [ 0.5445, -0.8721],
        [-0.0429,  0.9626],
        [-0.0891, -1.0896],
        [ 2.8788,  0.4289],
        [-0.6019, -0.1125],
        [ 0.0423,  0.8172],
        [ 0.3537, -0.1794],
        [ 0.9593,  0.9876],
        [-0.9673, -0.3296],
        [-0.6884, -0.8141],
        [-0.8447, -0.5433],
        [-0.7194,  1.5412],
        [ 0.3908,  1.0354],
        [ 0.2252, -0.5463],
        [-0.8880,  0.4946],
        [ 0.5082,  1.0557],
        [-0.0415,  1.4094],
        [-1.4365, -0.0553],
        [ 0.0622, -0.7926]], requires_grad=True)

Hidden Layer Biases:
Parameter containing:
tensor([0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
        0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.], requires_grad=True)

Output Layer Weights:
Parameter containing:
tensor([[-0.1823, -0.0738,  0.0201,  0.0296, -0.3859,  0.1555, -0.3157,  0.3774,
          0.0376,  0.5168, -0.2199, -0.0258,  0.0507,  0.0094, -0.0734,  0.6106,
          0.0462,  0.0753,  0.2163, -0.0321, -0.3229, -0.1537,  0.7679,  0.3028,
         -0.0864,  0.4478,  0.2042,  0.1297, -0.0255, -0.0764,  0.1590, -0.4226],
        [-0.3935,  0.0602, -0.1824,  0.2283,  0.0572, -0.0922,  0.1198,  0.1049,
         -0.3431, -0.1994, -0.2599,  0.1026, -0.4927, -0.0560,  0.1282, -0.3565,
         -0.1956,  0.0418, -0.3588, -0.3303, -0.1648, -0.1970, -0.1038, -0.2678,
         -0.0569, -0.0144, -0.1276,  0.3609,  0.2257,  0.4418,  0.1733, -0.0561],
        [-0.1553,  0.5354,  0.2328,  0.2689, -0.0444,  0.0818, -0.2989,  0.0496,
         -0.0652,  0.0915, -0.2709,  0.2607, -0.3689,  0.1295, -0.0652,  0.2961,
          0.3755, -0.0889, -0.0031, -0.1093, -0.0294,  0.1940,  0.3499, -0.3664,
         -0.2243, -0.0151,  0.0892, -0.1420,  0.2212, -0.0254, -0.0388, -0.1197]],
       requires_grad=True)

Output Layer Biases:
Parameter containing:
tensor([0., 0., 0.], requires_grad=True)
```

Manual, **very small initialization**!

```
MulticlassNoNLinearNNCustomW(
  (hidden_layer): Linear(in_features=2, out_features=32, bias=True)
  (output_layer): Linear(in_features=32, out_features=3, bias=True)
  (activation_function): ReLU()
)
Hidden Layer Weights:
Parameter containing:
tensor([[ 4.4154e-05, -2.8346e-05],
        [-9.6707e-05, -9.5724e-05],
        [-8.3815e-05, -3.3718e-05],
        [-1.5921e-05,  1.5213e-04],
        [ 1.0165e-04,  9.9832e-05],
        [-2.7500e-05, -2.6186e-05],
        [ 1.2590e-04,  1.3935e-04],
        [ 9.6503e-05,  1.8886e-06],
        [ 1.7044e-04,  1.3494e-04],
        [-8.3561e-05, -1.5529e-04],
        [-1.2645e-04, -5.1204e-05],
        [ 5.4781e-05,  3.3605e-05],
        [ 8.5806e-05, -2.3952e-05],
        [ 8.6836e-05,  5.4168e-05],
        [-1.4250e-04,  1.7874e-06],
        [-1.0494e-04, -1.0766e-04],
        [ 1.6171e-04,  6.2094e-05],
        [-1.0732e-04,  1.9195e-04],
        [ 1.8627e-04,  4.3418e-05],
        [-1.4717e-05,  4.5731e-05],
        [-1.4614e-04, -1.8345e-04],
        [-2.2154e-05, -3.5882e-05],
        [-1.4144e-06, -1.0451e-06],
        [-8.0837e-05, -1.3188e-04],
        [ 4.0737e-05,  9.4692e-05],
        [-1.8073e-04, -1.4635e-04],
        [ 1.4216e-04,  5.2984e-05],
        [ 3.4955e-05, -1.5466e-04],
        [ 8.6066e-05, -1.2416e-05],
        [ 2.4685e-05, -1.7295e-05],
        [ 9.3320e-05,  3.6173e-05],
        [-2.3997e-05,  5.0203e-05]], requires_grad=True)

Hidden Layer Biases:
Parameter containing:
tensor([-5.3260e-05, -2.1672e-04, -5.8774e-05,  1.6645e-04, -9.8405e-06,
        -9.0681e-05,  6.9803e-05,  3.8611e-05,  7.2955e-05,  1.2970e-05,
         1.2674e-04,  1.2268e-04, -6.2159e-05,  1.6131e-04, -1.0440e-05,
         4.9619e-05,  8.0752e-05, -1.2048e-04, -1.2480e-04,  2.4272e-06,
         2.0568e-05,  2.9018e-05,  3.7625e-05, -2.0461e-04, -5.2869e-05,
         3.5869e-05, -2.2357e-04, -1.8803e-05, -1.4234e-04,  6.6430e-05,
        -1.0538e-04,  7.9602e-05], requires_grad=True)

Output Layer Weights:
Parameter containing:
tensor([[-5.4834e-05,  1.2036e-04, -5.8962e-05,  6.1416e-06, -1.6622e-04,
         -5.8553e-05,  1.2438e-04,  7.2807e-05,  1.0892e-04,  5.1956e-05,
          1.8402e-04,  2.0608e-04, -5.9831e-05,  1.3045e-04, -1.5674e-05,
          7.7201e-06, -3.9317e-05,  1.6386e-05, -5.6655e-05, -1.7126e-06,
          1.6328e-04, -2.4760e-05, -1.8263e-04, -1.6302e-05, -2.4447e-04,
          6.6810e-05, -1.0898e-04, -4.4622e-06,  2.1247e-05, -5.9792e-06,
          1.2561e-04, -1.5936e-05],
        [-7.8843e-05, -3.5543e-05,  6.5130e-05,  5.2078e-05,  7.6328e-06,
         -9.1079e-06,  1.4718e-04, -1.1352e-04, -1.2195e-04,  2.8074e-05,
         -6.6907e-05,  1.7270e-04, -1.4943e-04, -3.8431e-05, -1.3685e-05,
          1.3792e-04,  6.8393e-05,  4.5187e-05,  1.4251e-04, -5.1791e-05,
         -4.1046e-05, -3.8371e-05, -1.1805e-04,  1.2429e-04, -5.7743e-05,
         -1.0395e-04,  5.5444e-05,  6.7977e-05, -5.5835e-05,  8.1546e-05,
         -8.2545e-05, -3.5153e-05],
        [-3.5222e-05, -3.2911e-05,  7.1239e-05, -2.7563e-05, -1.3171e-05,
         -5.0729e-05, -2.0779e-04, -5.9498e-05, -6.0210e-05, -4.6387e-05,
          6.4751e-06, -1.6025e-04, -1.4051e-04,  5.4637e-05, -7.9265e-06,
          2.5656e-05, -1.8643e-05, -3.5562e-05, -1.4493e-04, -1.9958e-05,
          1.4264e-05,  1.6963e-05,  4.7444e-05,  2.8374e-04,  7.2045e-05,
         -5.4671e-05, -5.8124e-05, -1.1368e-05,  1.9649e-05, -2.1953e-04,
          9.9693e-05,  1.5244e-05]], requires_grad=True)

Output Layer Biases:
Parameter containing:
tensor([-9.1749e-05, -5.5175e-06,  1.1487e-04], requires_grad=True)
```

MISCELLANEOUS

- **Device to use the architecture:** Deep Learning works well on GPUs!
  - That are frequently not available ☹
    - Or with limited access
    - Use it only when required
  - Small dataset may not require GPUs ☺
  - Device = 'cuda' tells to use the GPU if available!
  - To send the model/data to the device we can use the *.to(device)* function
  - **NOTICE: if you use 'cuda'** use .to(device) to load the data on gpu (before) and dump to cpu (after)
    - Not loading ➔ You do not use the GPU memory – not optimize
    - Not dumping ➔ You may run out the GPU memory
- **Help Reproducibility**: Instead of manual assign weights set the random seeds of all function that can have a random behaviour
  - Weights initialization, random number, validation split
  - WARNING: If you use different Hardware (e.g., in Colab) **you may still get different results** ☹

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

```
# Move model to device
model = model.to(device)
```

```
for batch_X, batch_y in train_dataloader:
    batch_X, batch_y = batch_X.to(device), batch_y.to(device)
```

```
import random
import torch
import numpy as np
from sklearn.model_selection import train_test_split

seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)


X_train, X_val, y_train, y_val = train_test_split(X_t, y_t, test_size=0.2, random_state=seed,
stratify=y_t)
```

DEMO

Play with an interactive FFNN: https://playground.tensorflow.org/

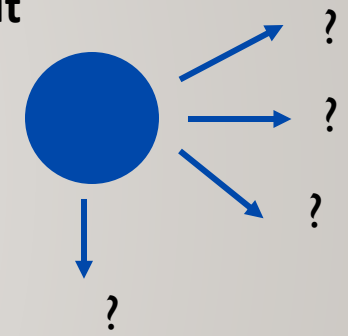
And try with PyTorch!

RECURRENT NEURAL NETWORK (RNN)

Theory

WHY DO WE NEED RNN?

- Standard NN models do not handle sequences of data
  - They accept a **fixed-sized vector** as **input** and produce a **fixed-sized** vector as **output**
    - The **weights** are updated **independent** of the **order** the samples are processed
- Using a FFNN can you can you predict the next position???



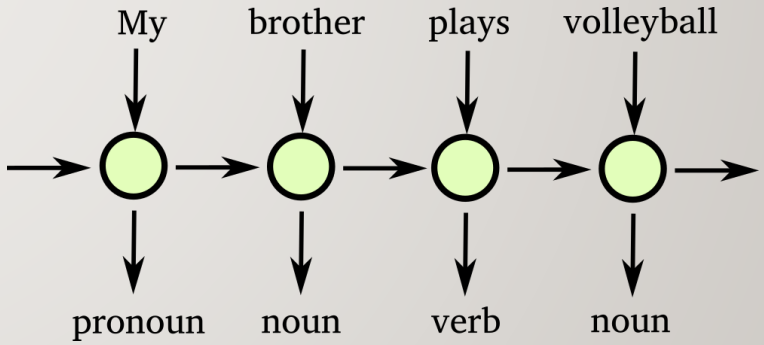
➔ Recurrent Neural Networks (RNN)s are designed for modelling sequences

- What if I give you the sequence of previous movements… and a RNN that handles sequences?

Image taken from *introtodeeplearning.com*

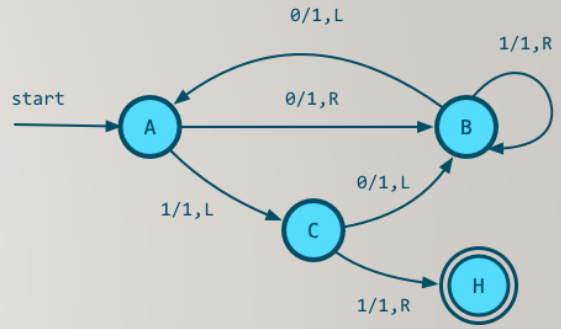
RECURRENT NEURAL NETWORKS

- A class of neural networks allowing to handle **variable length inputs**
- Allow processing *sequential* data x(t)
- Differently from normal FFNN they are able to keep a *state* which evolves during time
- Applications
  - machine translation
  - **Time-series prediction**
  - speech recognition
  - part of speech (POS) tagging



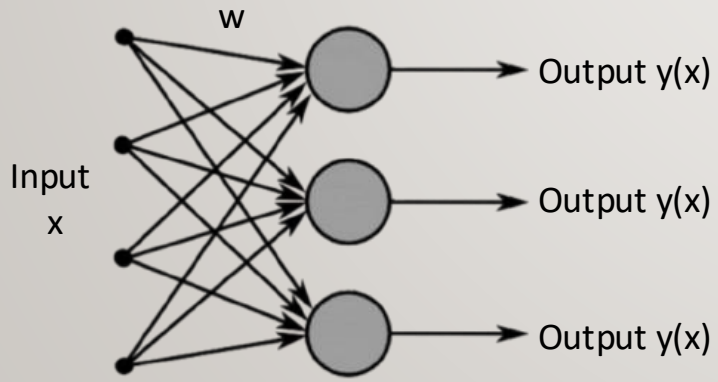
REPRESENTATIONAL POWER OF RNNS

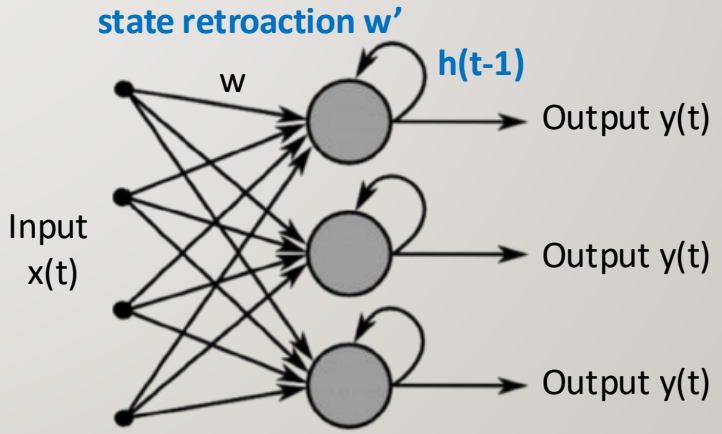
- Regular feed-forward neural networks are not **Turing-Complete**
  - They can represent **any single mathematical** function but **do not have** any ability to perform **looping** or other control flow operations
  - RNNs are Turing-Complete: they can simulate **arbitrary programs**
- They can represent any mapping between input and output sequences ➔ versatility



FFNN VS RNN IN A NUTSHELL

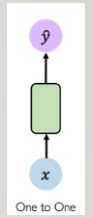
- A FFNN receives as input a vector x and each gives the output(s) y
- A RNN receives as input a vector x(t) and gives
  - The output y(t)
  - **And** the **hidden state** at **previous time step** h(t-1)
  - All weights w and w' must be computed **during the training** process
- A RNN typically contains many *neurons organized in different layers*

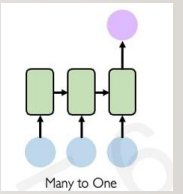


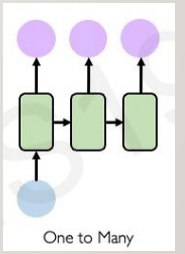


RNN ARCHITECTURES: INPUT/OUTPUT VARIATIONS SEQUENCE MODELLING APPLICATION

- **One-to-One**: Standard neural network processing one time-stamp per time
  - Used in simple classification and regression tasks
  - E.g., Regression using a single sample per time
- **Many-to-One**: Processes a sequence of inputs, producing a single output
  - The final hidden state encodes the entire sequence
  - E.g., sentiment analysis, analyse sequence of log entries to detect an ongoing cyberattack
- **One-to-Many**: Single input generates a sequence of outputs
  - The initial input is expanded over multiple time steps.
  - E.g., image captioning, generate an incident response plan from an initial threat detection alert
- **Many-to-Many**: Sequence-to-sequence mapping, where input and output lengths may vary (e.g., machine translation).
  - E.g., translation, Detecting anomalies in real-time network traffic, where each input frame is classified







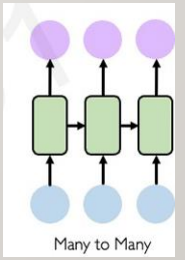
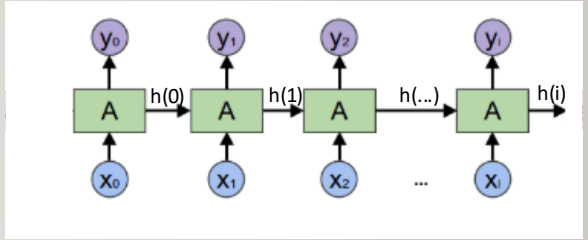


Image taken from *introtodeeplearning.com*

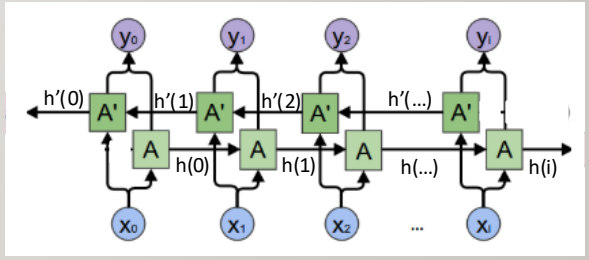
RNN ARCHITECTURES

1. **Mono-directional RNN**: Process data sequentially, updating a hidden state at each step using the current input and previous hidden state
   - **PROS**: Efficient for real-time prediction tasks
   - **CONS**: Can struggle with long-term dependencies due to vanishing/exploding gradients
   - **TRAINING**: Backpropagation **Through Time** (**BPTT**), a variant of backpropagation for sequential data
   - **APPLICATIONS**: Supervised network classification as **benign** or **malicious, unsupervised** anomaly detection in network traffic



Source: colah's blog

2. **Bi-directional RNN (Bi-RNN):** Bi-RNNs process data in both forward and backward directions, maintaining two hidden states. How? **Two RNNs stacked on top of each other**. When we know the full trajectory.
   - **PROS**: Captures both past and future context, e.g., for speech processing.
   - **CONS**: Higher computational cost and not suitable for real-time prediction – **no future is** available
   - **TRAINING**: BPTT is applied **separately** for both forward and backward directions, and their outputs are later combined
   - **APPLICATIONS**: Supervised email classification to detect phishing attempts based on linguistic and structural patterns, Log File Pattern Recognition for Insider Threat Detection



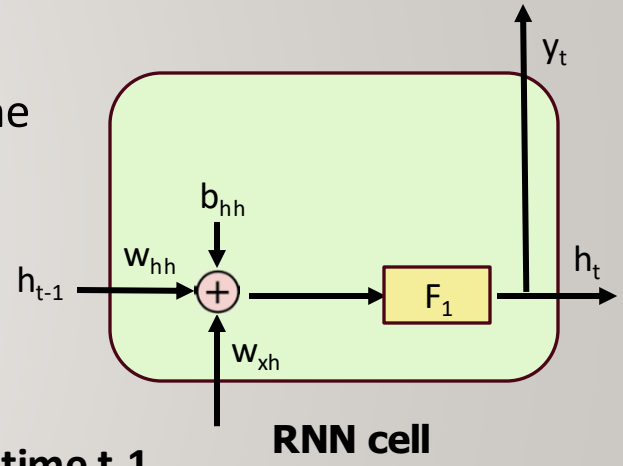
Source: colah's blog

RNN ARCHITECTURES: INTERNAL STRUCTURE

- A Recurrent Neural Network (RNN) processes sequential data by maintaining a hidden state h_t, which carries information from previous time steps.

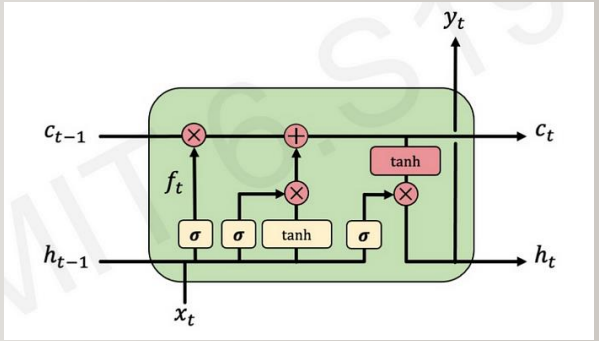
At each time step t, an RNN node updates its **hidden state h_t** and computes the **output y_t**:

- $h_t = F_1(W_{hh}h_{t-1} + W_{xh}x_t + b_{hh})$
  - h_{t-1} is the **previous hidden state**
  - x_t is the **current input data**
  - W_{xh} and W_{hh} are the **weight matrices** of the the **input** and the **hidden state at time t-1** respectively
  - b_{bh} is **the bias term (weight)** related to the **hidden state**
  - F_1 is the **activation function** (Typycally **Tanh – not ReLU**, to address possible gradient problems)
- The outuput y_t **of the RNN** cell is equal to h_t



RNN ARCHITECTURES

3. **Long Short-Term Memory (LSTM)**: RNN that solves the vanishing gradient problem using gates (input, forget, and output gates) to regulate the flow of information.
   - **PROS**: Handles long-range dependencies effectively.
   - **CONS**: Computationally expensive compared to vanilla RNNs.
   - **TRAINING**: Still trained using BPTT, but the gating mechanism helps mitigate gradient issues
   - **APPLICATIONS**: When remember long sequence is important



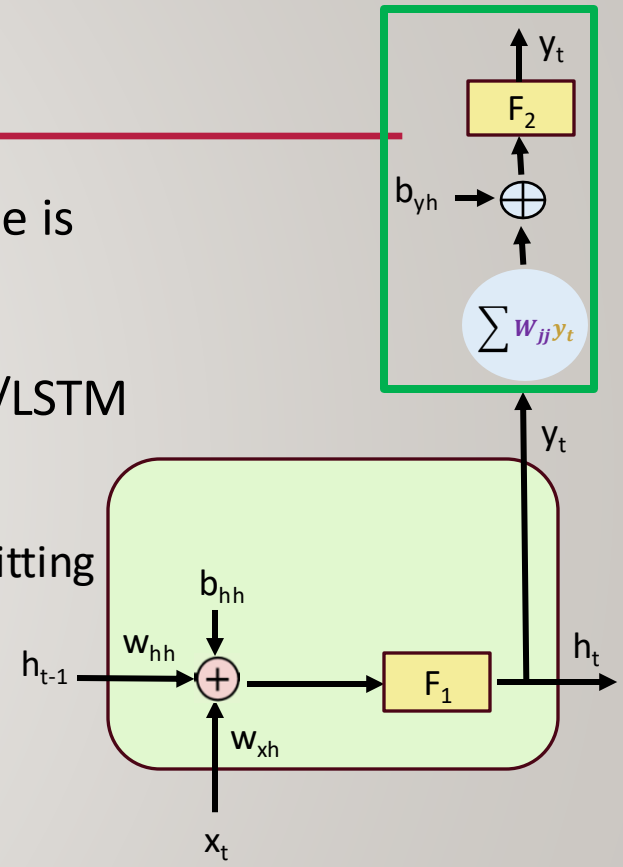
LSTM cell

Image taken from *introtodeeplearning.com*

RNN ARCHITECTURES: PREDICTION

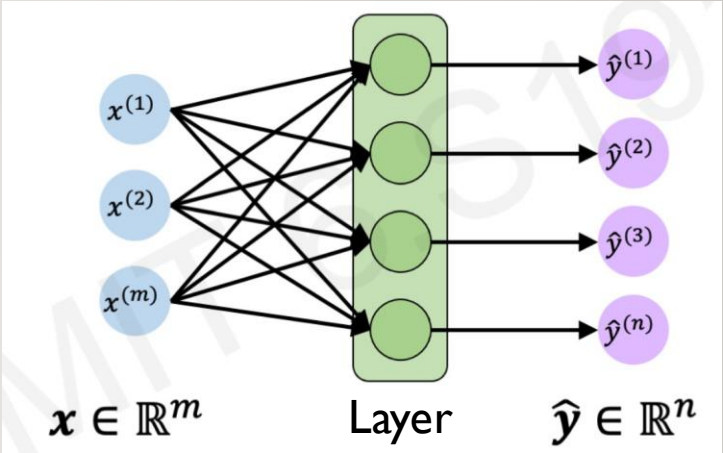
Y_t it is composed by multiple values for each time x(t). The number of value is the size of the hidden/cell states. How to make the final prediction??

- To compute the **final prediction** we take the out y_t or h_t of the **last** RNN/LSTM **node** and add (at least) a **new linear layer**
  - We may additional dropout/normalization layer can be add to reduce ovefitting
  - Adding an activation function (if required)
- we compute $y_t = F_2(W_{yh}\boldsymbol{h_t} + b_{yh})$
  - h_t is the computed **hidden state** at time **t**
  - W_{yh} is the **weight matrix** related to the **output y**
  - b_{yh} is the **bias term (weight)** related to the **oput computation**
  - F_2 is the final **activation function** (if required)



FEED-FORWARD NEURAL NETWORKS REVISITED

- Classic FFNN with a single layer and multiple outputs
  - No notion of temporal sequence yet



- Same NN architecture with a different representation
  - Now the input x_t is an input that we demonstrate a specific time *t*

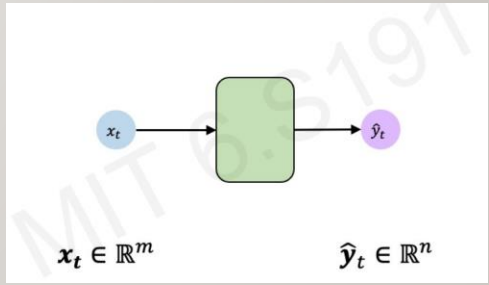


Image taken from *introtodeeplearning.com*

HANDING INDIVIDUAL TIME STEPS

- Take the same NN architecture and apply it **several time**
  - **Input** is still a **vector** of **numbers**
    - **Time independent**
  - Each network is independently with respect to the previous one
    - C**omputing** the **output independently step by step**
    - There is no concept of sequence!
    - The error is computed for each network separately
    - Input x_0 and x_1 di not influence $\hat{y}_2$

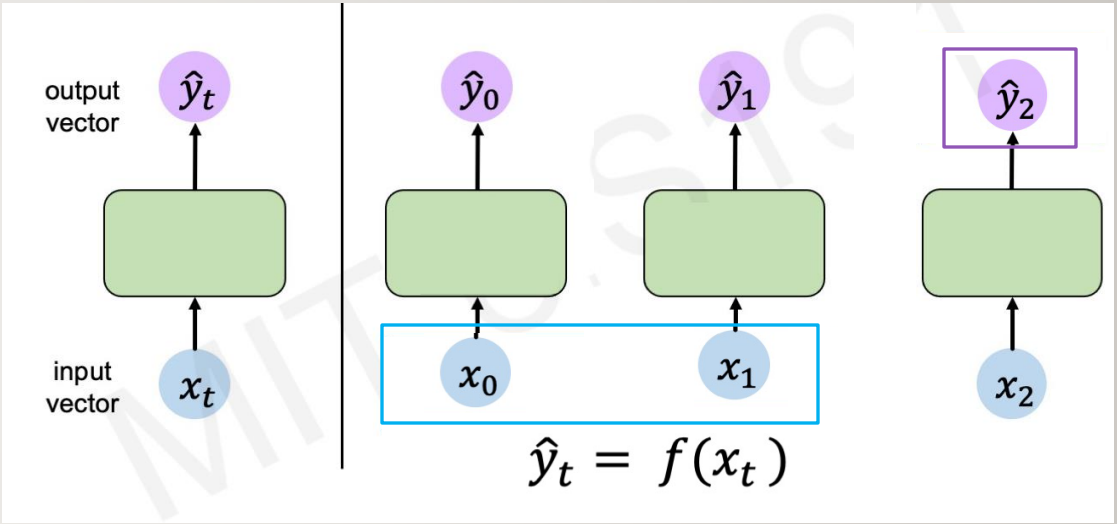


Image taken from *introtodeeplearning.com*

HANDING INDIVIDUAL TIME STEPS

- Why not chain these NN to consider that there is temporal sequence and temporal and temporal relation among inputs at different x_t??

➔ Connect each NN with a link!

- h_t is the **state** of the **neural network** and is computed by it and propagated over time at each neuron

➔The **output function** $\hat{y}_t$ **changes** considering the temporal state asl well!!

- Depends on the input
- The past memory

- Abstract version of our NN with recurrent
  - ➔ Create a Recurrent NN composed by **Recurrent Cells**
  - ➔ The **right** NNs are the unrolled visualization of the left **RRN**

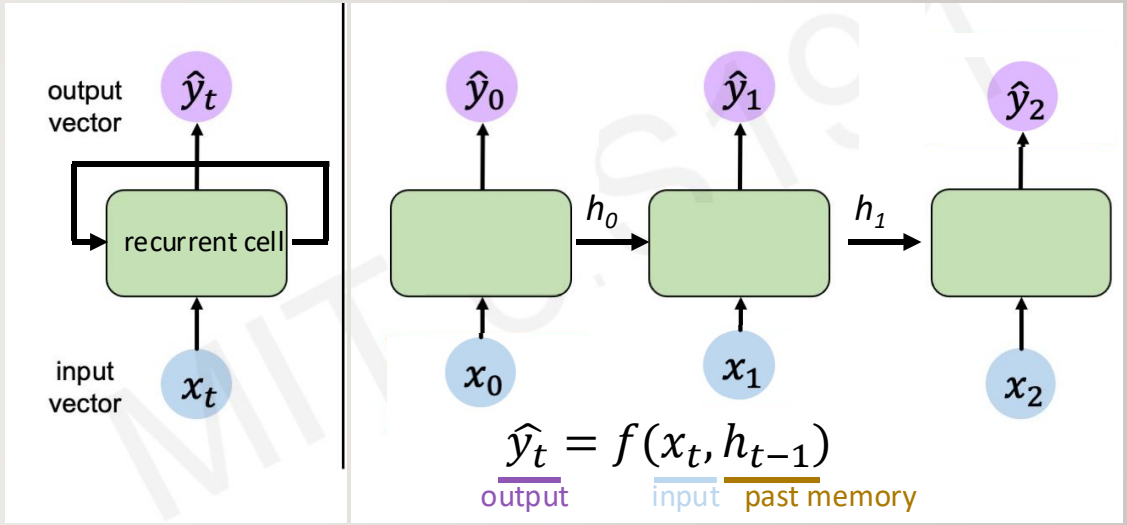


Image taken from *introtodeeplearning.com*

FORMALIZATION: RECURRENT NEURAL NETWORK (RNNS)

- Formalization of the foundation of the sequence modelling!
  - **State h_t** while **updated** at **each time step** while we process the sequence
  - ➔Apply a recurrent relation at every time step to process a sequence
    - ➔ h_t: learn a set of weights W in function of x_t (input in each time step) and h_{t-1} (old state)
- For a particular RNN layer we have the same weights of parameters while the model is learned!
  - Same functions, same set of weights
  - What is the difference? Now we evaluate one time step at the time

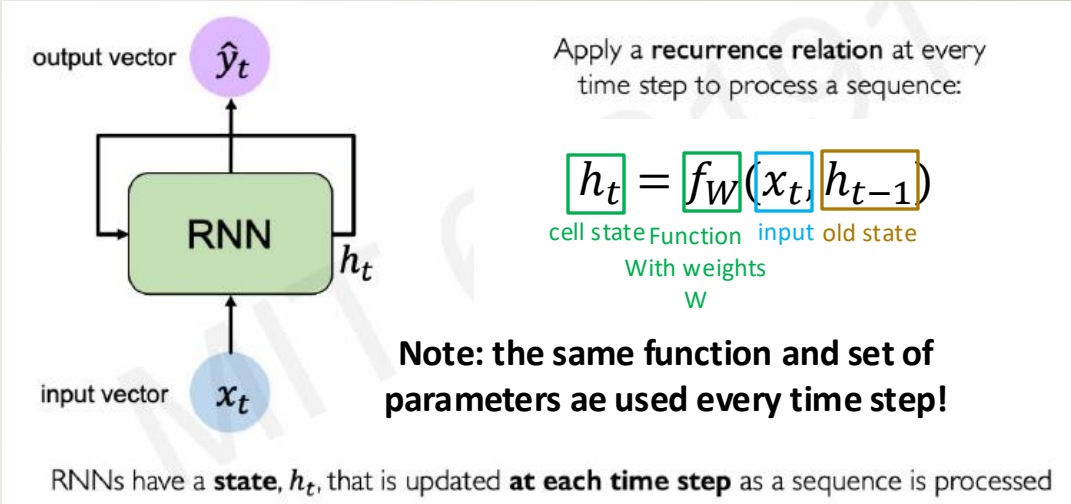
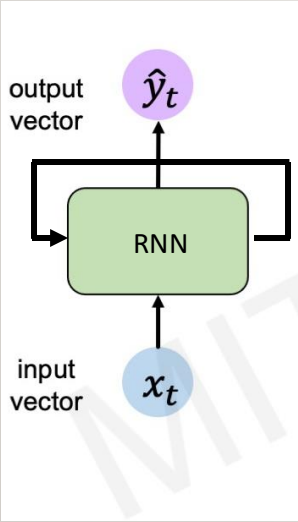


Image taken from *introtodeeplearning.com*

RNNS: COMPUTATIONAL GRAPH ACROSS TIME

1. Representation of the RNN
   - Compact where the full NN is described by a single node having a loop **representing the hidden state**
   - This represent the full architecture with the input varying every time t and producing an output



= Represents as computational graph enrolled across time

Image taken from *introtodeeplearning.com*

RNNS: COMPUTATIONAL GRAPH ACROSS TIME

2. Unroll RNN representation
   - Shows how at every timestamp the input x_t at time *t* enters the network producing an output y_t
   - $\boldsymbol{W_{xh}}$: This matrix maps the input x_t into the the hidden state h_t
   - $\boldsymbol{W_{hh}}$: This matrix defines how the previous hidden state h_{t-1} influences the current hidden state h_t
   - $\boldsymbol{W_{hy}}$: This matrix maps the hidden state h_t into the output value y_t
     - **ALL** these **matrices** are computed and updated every time-stamp during the training process

Notice: After training the RNN re-use the **same weight matrices** at every time step!

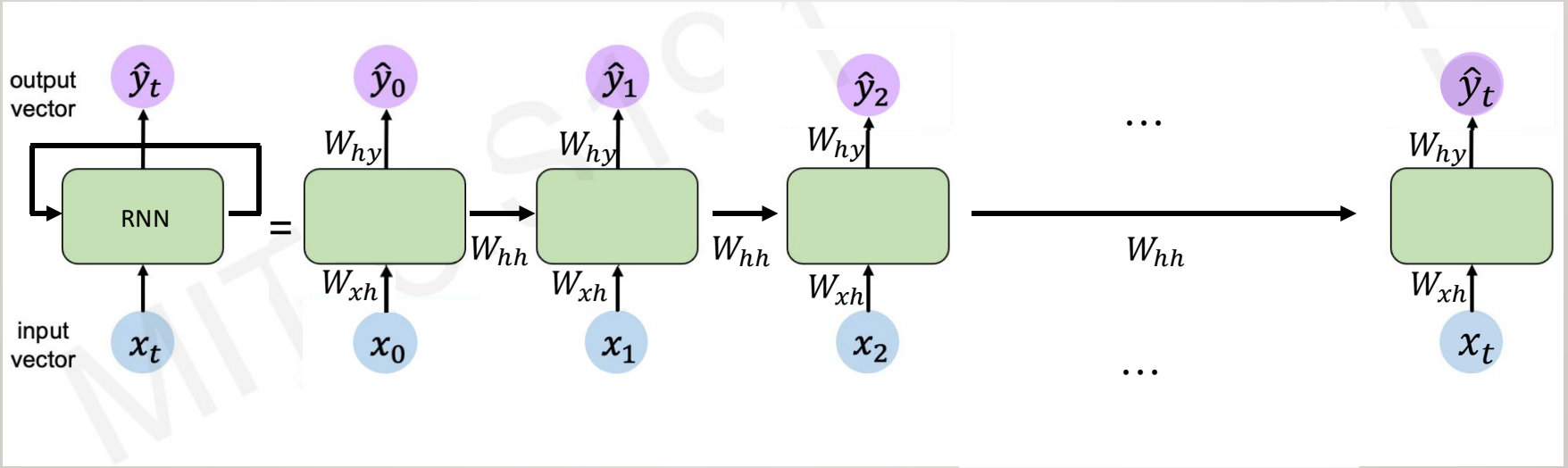


Image taken from *introtodeeplearning.com*

RNNS: BACKPROPAGATION THROUGH TIME

- To learn the weights, we have to use the back probation as before..
  - … But now we have the time
- So we sum all Loss values to compute the Total Loss across all sequence

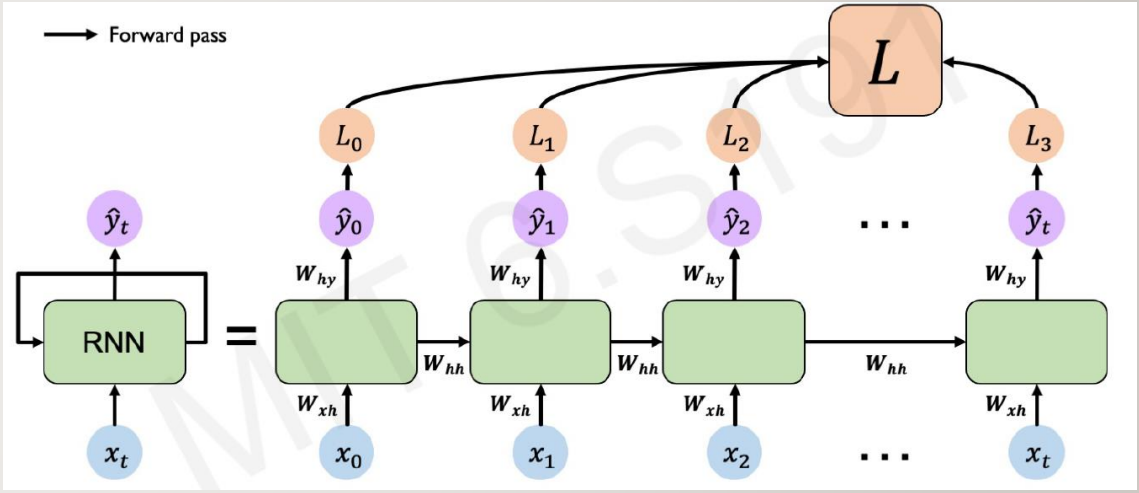


Image taken from *introtodeeplearning.com*

WRAPPING UP: RRN STATE UPDATE AND OUTPUT

- We can update at each time h_t the state variable accordingly to input vector x_t
- This math function describes how to update the hidden state h_t
  - Similar wr.t. the previous matrix for FFNN
  - Nonlinear function (*tanh*)
  - Learning the W_{xh}^T **matrix** for updating the **weights** of the **input**
  - Learning the W_{hh}^T **matrix** for updating the **weights** of the **hidden states**
  - **Recall: h_t** is per neuron
    - Each RNN layer can have multiple neurons based on the layer size
- Finally compute the output in function of the hidden state

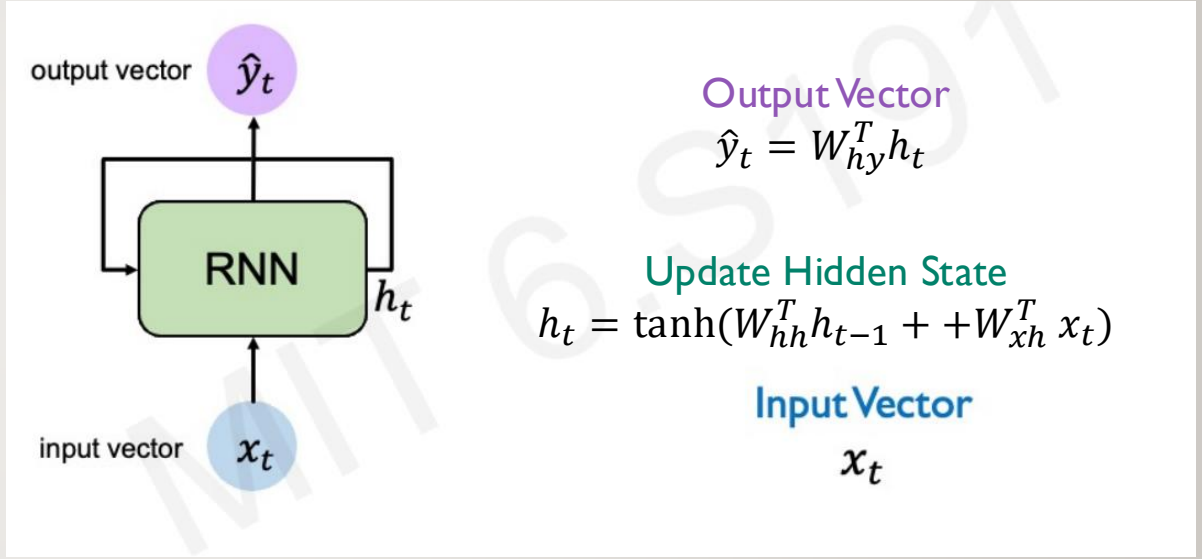


Image taken from *introtodeeplearning.com*

RECALL: BACKPROPAGATION IN FEED-FORWARD NEURAL NETWORKS

- Classic backpropagation **algorithm**

1. Forward the samples through the network and compute the loss
2. Backpropagate the error taking the derivative (gradient) of the loss with respect to each parameter
3. Update the internal parameters (weights and biases ϕ in order to minimize the loss

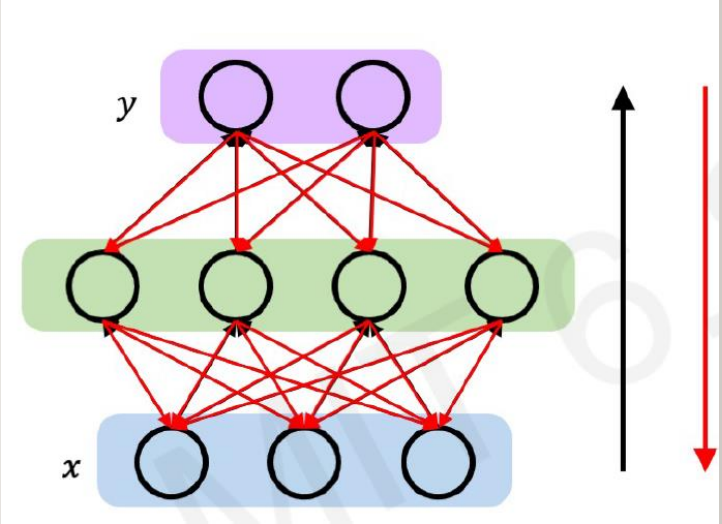


Image taken from *introtodeeplearning.com*

RNNS: BACKPROPAGATION THROUGH TIME

- With RNN the time makes it more complicated
  - *Paul Werbos,* Backpropagation Through Time*: What It Does and How to Do It, Proceedings of IEEE, 78(10):1550-1560, 1990*
- We send each loss L_1 through time to compute a total loss (L)

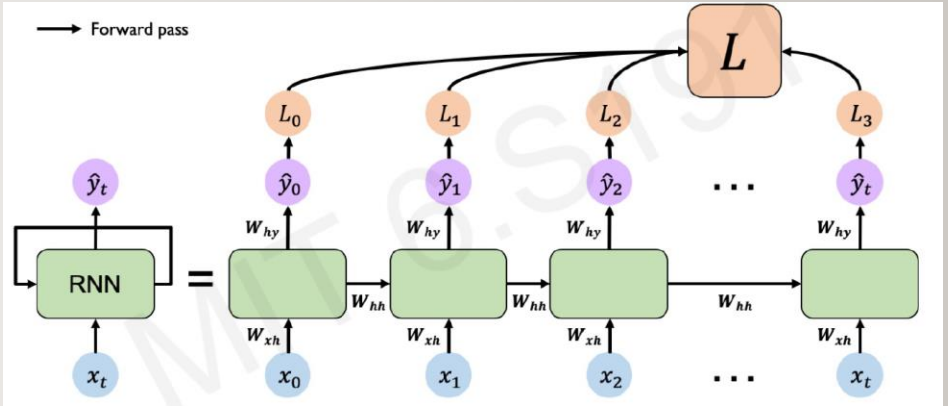


Image taken from *introtodeeplearning.com*

RNNS: BACKPROPAGATION THROUGH TIME

1. We have to backpropagate the error and adjust the weights through the different time step individually
2. For the end to the beginning of the sequence!

➔ Backpropagation through time idea!

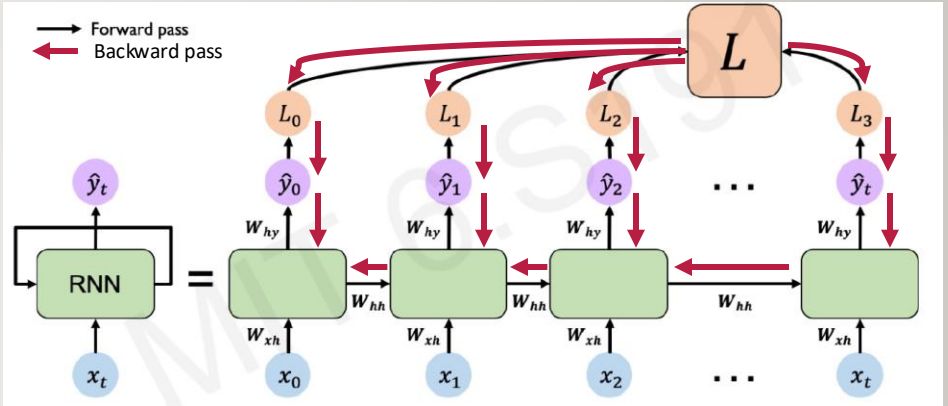


Image taken from *introtodeeplearning.com*

STANDARD RNN GRADIENT FLOW

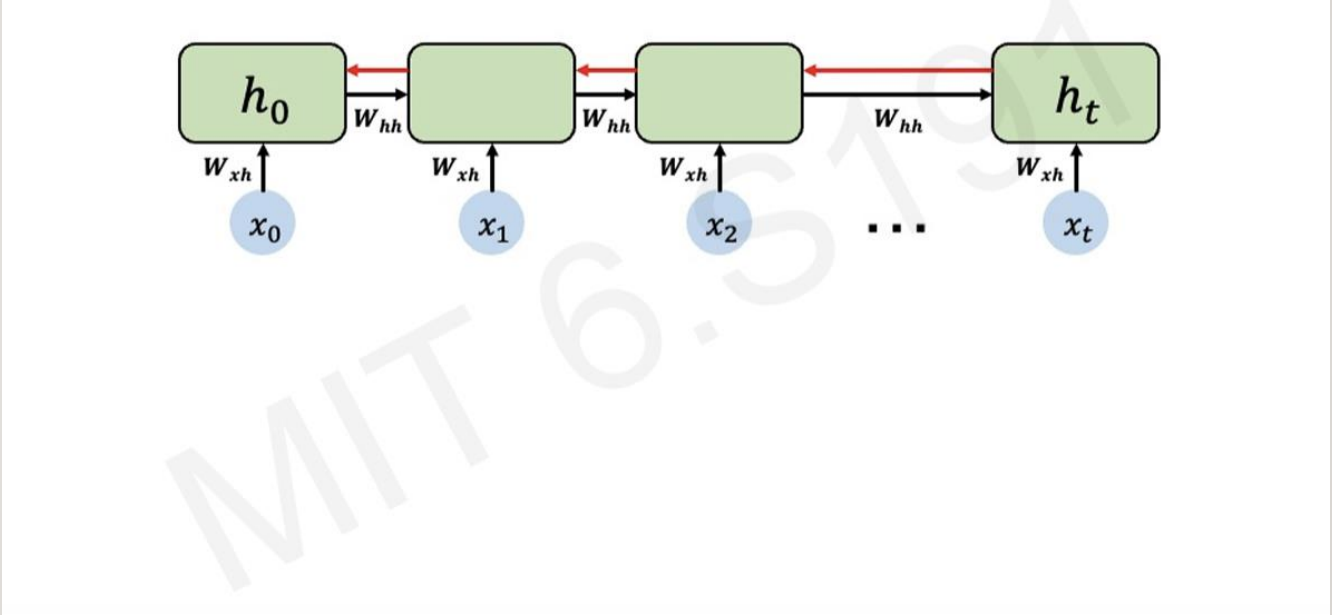
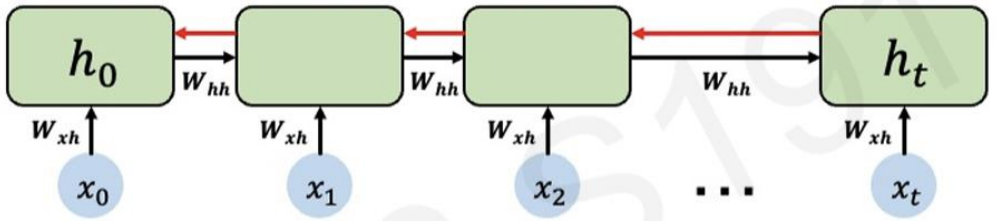


Image taken from *introtodeeplearning.com*

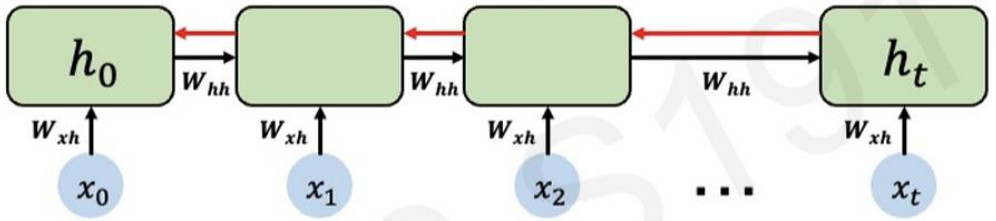
STANDARD RNN GRADIENT FLOW



Computing the gradient wrt h_0 involves **many factors ow W_{hh} + repeated gradient computation**!

Image taken from *introtodeeplearning.com*

STANDARD RNN GRADIENT FLOW



Computing the gradient wrt h_0 involves **many factors ow W_{hh} + repeated gradient computation**!

Many values > 1:
Exploding gradients

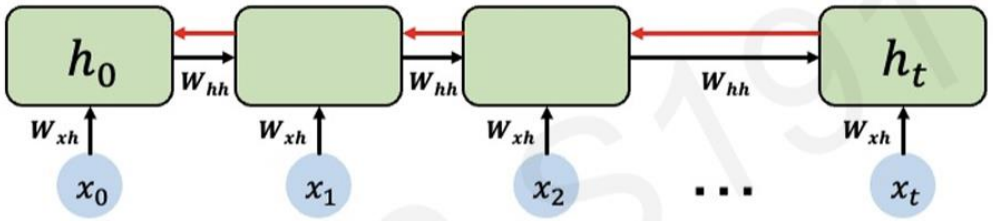
Gradient clipping to scale big gradients

- As a Result the network will not be able to learn!

➔ This is why we use Tanh instead of ReLU!

Image taken from *introtodeeplearning.com*

STANDARD RNN GRADIENT FLOW



Computing the gradient wrt h_0 involves **many factors ow W_{hh} + repeated gradient computation**!

Many values > 1:
Exploding gradients

Gradient clipping to scale big gradients

Many values < 1:
Vanishing gradients

1. **Activation function**
2. **Weight initialization**
3. **Network architecture**

- As a Result the network will not be able to learn!

➔ This is why we use Tanh instead of ReLU!

- Opposite the gradient shrink and the network will not be able to learn

Image taken from *introtodeeplearning.com*

THE PROBLEM OF LONG-TERM DEPENDENCIES

- The problem of long term dependents born when we have to study sequences composed by many temporal moments
  - E.g., if the series is a book use all the words of the book together

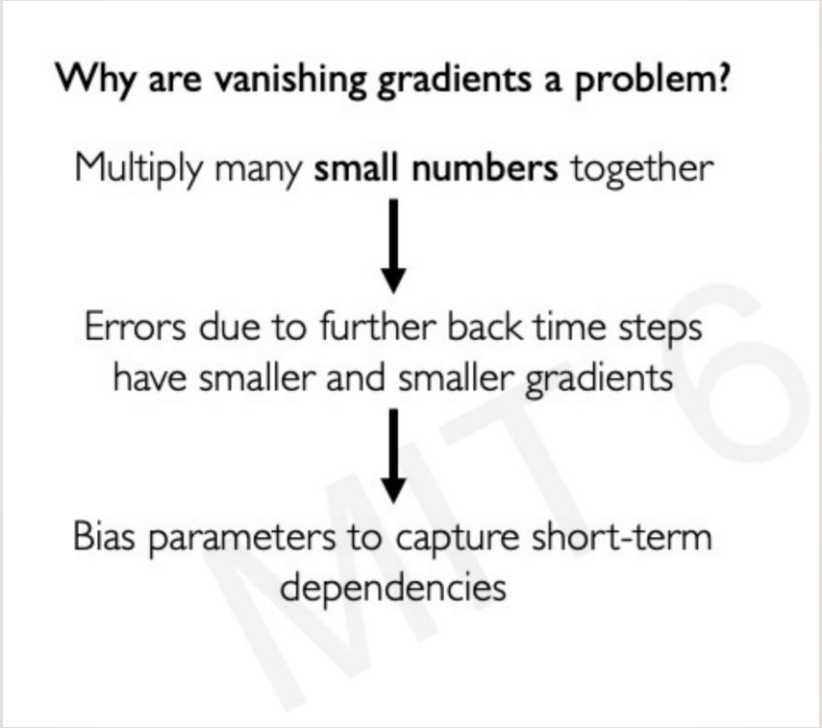


Image taken from *introtodeeplearning.com*

THE PROBLEM OF LONG-TERM DEPENDENCIES

- We should have enough memory capacity to store previous story

  ➔ Able to learn from the past to predict the future

Why are vanishing gradients a problem?

Multiply many **small numbers** together

↓

Errors due to further back time steps have smaller and smaller gradients

↓

Bias parameters to capture short-term dependencies

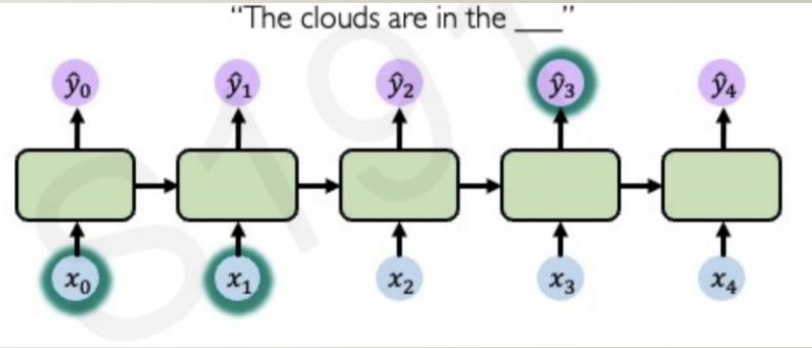


Image taken from *introtodeeplearning.com*

THE PROBLEM OF LONG-TERM DEPENDENCIES

- What if our memory is too short to remember the past?

Why are vanishing gradients a problem?

Multiply many **small numbers** together

↓

Errors due to further back time steps have smaller and smaller gradients

↓

Bias parameters to capture short-term dependencies

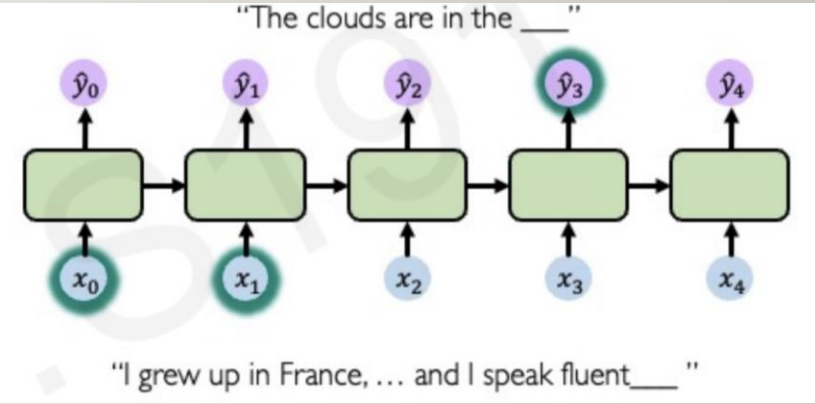


Image taken from *introtodeeplearning.com*

THE PROBLEM OF LONG-TERM DEPENDENCIES

- The network capacity to learn and remember the future may be **destroyed**
  - We cannot remember to far in time

Solutions:

1. Use a proper activation function!
   - i.e., the Relu
     - Everywhere?
2. Initialize the weights properly to have a good starting point avoid gradient vanishing problem

Why are vanishing gradients a problem?

Multiply many **small numbers** together

↓

Errors due to further back time steps have smaller and smaller gradients

↓

Bias parameters to capture short-term dependencies

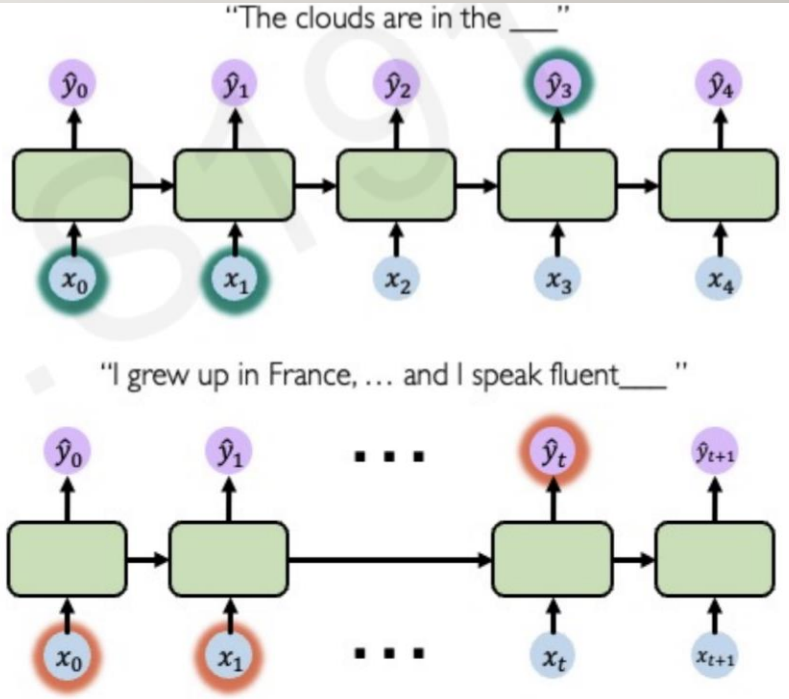


Image taken from *introtodeeplearning.com*

MORE ROBUST SOLUTION: CREATE A SMARTER NEURON A GATED CELL

- **Gating**: Introduce additional computation
  - The gates are used to understand what we should remember and what we should forget!
- Different architectures, LSTM, GRU, etc.
  - **We will focus on LSTM**

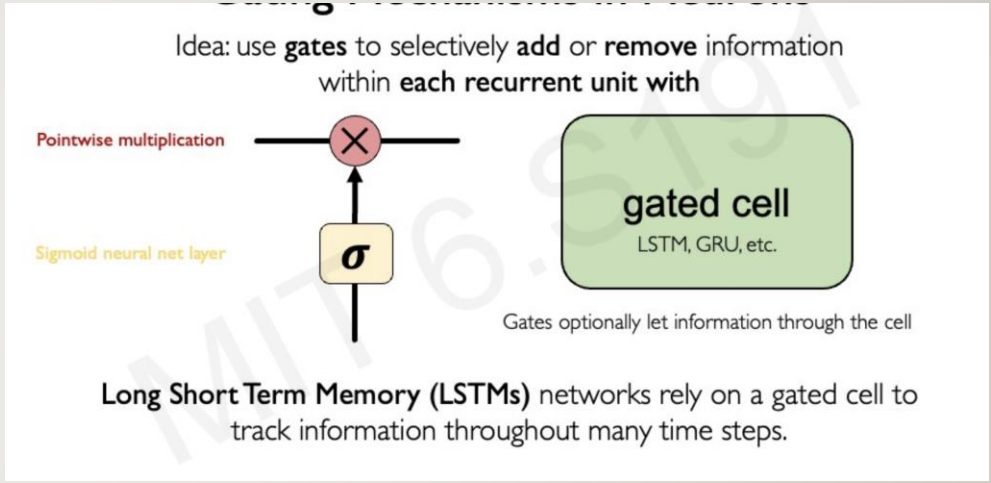
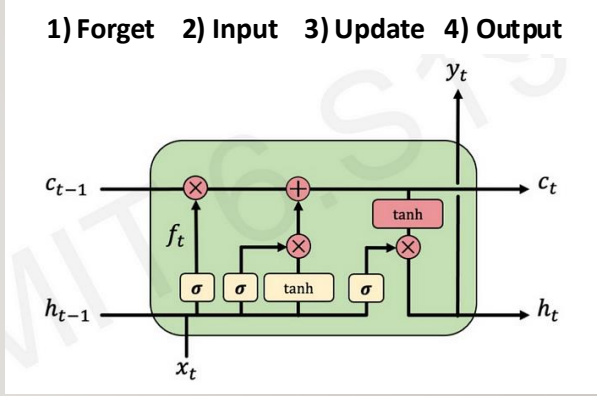


Image taken from *introtodeeplearning.com*

LSTM (LONG SHORT-TERM MEMORY): KEY IDEA



- Suggested in 1997 by Hochreiter and Schmidhuber as a solution to the vanishing gradient problem
- An LSTM cell stores a value (state) for either long or short time periods

1. Maintain the hidden state **and a cell state**
2. **Use gates** to control the flow of information
   - **Store relevant** information from current point
   - Selectively **update** cell state
   - Output gate returns a filtered version of the cell state
3. Backpropagation through time with partially **uninterrupted gradient flow**

Image taken from *introtodeeplearning.com*

LSTM: FORGET GATE

Key Idea:

- The **Forget Gate** decides **how much past information** to keep or discard from the **cell state**
- It uses a **sigmoid activation** to output values between
  - **0 (forget completely)** and **1 (keep everything)**

How It Works:

1. Takes **previous hidden state** (h_{t-1}) and **current input** (x_t).
2. Computes $\mathbf{f_t}$ a value between 0 and 1 using a sigmoid function
3. Multiplies $\mathbf{f_t}$ with the **previous cell state** (C_{t-1}), deciding how much memory to retain.

Example:

- $f_t \approx 1$, the LSTM remembers past information
- $f_t \approx 0$, the LSTM forgets past information

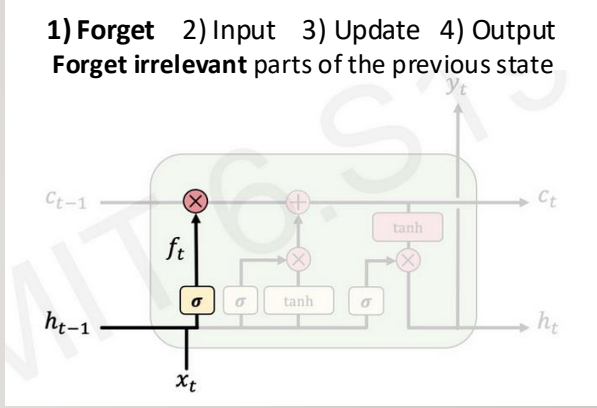
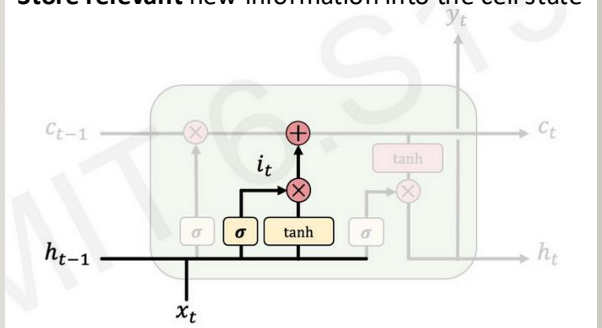


Image taken from *introtodeeplearning.com*

LSTM: INPUT GATE

1) Forget **2) Input** 3) Update 4) Output

Store relevant new information into the cell state



Key Idea:

- The **Input Gate** decides **what new information** should be added to the **cell state** (c_t).
- Uses a **sigmoid activation** to filter relevant information and a **tanh activation** to generate new candidate values

How It Works:

1. Takes **previous hidden state** (h_{t-1}) and **current input** (x_t).
2. Determines **how much new information to store** using i_t.
3. **Creates a candidate memory** $\tilde{C}_t$ and **updates** the cell state

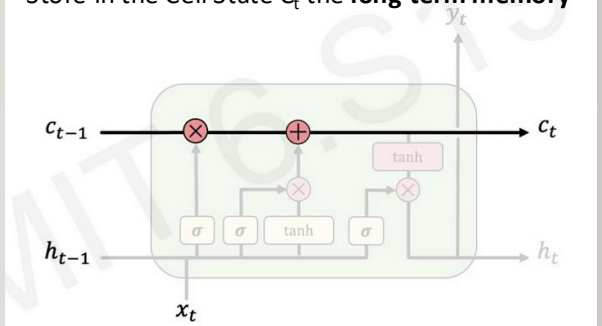
Example:

- $i_t \approx 1$, the new information is **strongly stored**.
- $i_t \approx 0$, the new information is **ignored**.

Image taken from *introtodeeplearning.com*

LSTM: UPDATE GATE

1) Forget 2) Input **3) Update** 4) Output
Store in the Cell State C_t the **long-term memory**



Key Idea:

- The Update or **Cell Gate** generates a **new candidate memory** that could be stored in the **cell state** (C_t).
- It works **alongside the Input Gate** to decide **what information to add** to the memory

How It Works:

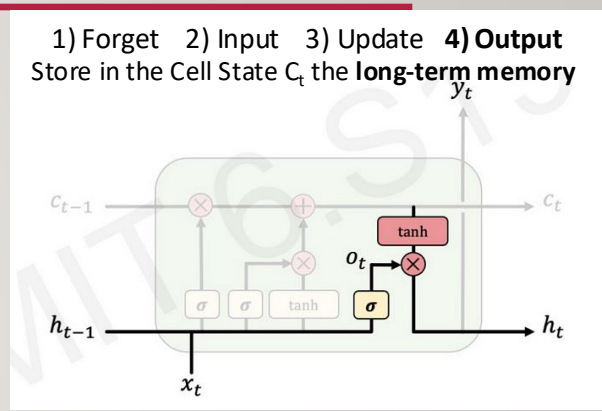
1. Takes the **previous hidden state** (h_{t-1}) and **current input** (x_t).
2. Generates a **candidate memory update** ($\widetilde{C_t}$) using **tanh activation**.
3. The **Input Gate** (i_t) determines how much of $\widetilde{C_t}$ should be added to the final **cell state**

Example:

- If $\widetilde{C_t} \approx 1$, it strongly **adds new information**.
- If $\widetilde{C_t} \approx 1$, **no significal updates** occurs.

Image taken from *introtodeeplearning.com*

LSTM: OUTPUT GATE



Key Idea:

- The **Output Gate** (o_t) controls how much of the **current cell state** (C_t) contributes to the **hidden state** (h_t).
- This determines what information is **passed to the next layer** or **used for predictions**.

How It Works:

- **Receives input** from the previous hidden state (h_{t-1}) and current input (x_t).
- **Computes o_t** using **sigmoid activation**, deciding how much memory to output.
- **Applies o_t** to a **tanh-transformed cell state C_t**, producing the **hidden state** h_t.
- **h_t** is then used to **predict y_t** or passed to the next time step
  - y_t=activation($W_y h_t + b_y$). b_y is a **bias term** to adjust the prediction

Example:

- If $o_t \approx 1$, most of C_t influences h_t
- If $o_t \approx 0$, very little of C_t affects the output

Image taken from *introtodeeplearning.com*

RECURRENT NEURAL NETWORK (RNN)

How?

RECURRENT NEURAL NETWORK: HOW TO INPUT DATA?

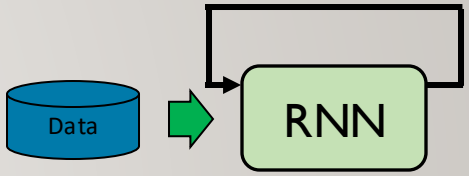
How to **preprocess** and **transform** the data:

- Data must be **numbers and series** ➔ The data must represent a **sequence** by means of a 3-D **matrix**!
  - E.g., [[[1,2,3],[4,5,6]], [[7,8,9],[10,11,12]]]
  - Each **vector** (e.g., [1,2,3]) is a **moment** in time **x(t)** and each **element** is a **features** of x(t)

    i.e., in this case a multi-dimensional time series problem
  - Each **matrix** (e.g., [[[1,2,3],[4,5,6]]) represents time series that the RNN must study reclusively
    - I.e., in this case 2 points in time described by 3 features each
  - All matrices together represent all time series
- What if data is a mono-dimensional time-series?
  - E.g., [[1],[2],[3]]
  - Same matrix representation ➔ Differently from FFNN now we work with matrices!
- What if time-series in the data have different dimension?
  - Is it a real problem?
  - How?
    - Add some padding/cut the time-series to have all of them of the same length

RECURRENT NEURAL NETWORK: WHAT ABOUT THE MODEL?

The **architecture**

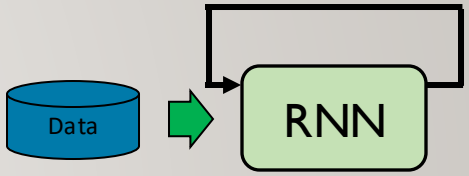


1. **Type:** what type of RNN do I have to use?
   - Input-output relationships: **many-to-one, one-to-many, many-to-many**
   - How long are the sequences: RNN, LSTM, …?
   - Learning backward: Mono-directional or Bi-directional?
2. **Input layer**: how many nodes?
   - Based on the number of features after preprocessing and transformation
3. **Hidden-size**: How many features to represent features in the hidden state h and cell state c (LSTM)?
4. **Hidden-layers**: How many?
   - Start with default and increase if required
   - Which activation function **per layer**?
     - Typically, tanh
5. **Padding**: Is there some padding in the time-series?
   - Do not consider the padding as real part of the time-series
6. Any architecture before the **output layer**?
   - Use a simple single linear layer used as output layer?
7. **Output layer**: how many?
   - Based on the number of classes
   - Which activation function?
     - Depending on the problem/Loss function, e.g., Sigmoid, SoftMax, ..?

RECURRENT NEURAL NETWORK: HOW TO OPTIMIZE IT?

How to optimize the architecture

5. How to input the data?
   - Shuffle mini-batches?
6. Which Loss Function?
   - If unbalanced, should we consider weights?
7. Which Optimizer?
   - How many epochs and what learning rate?



In case of overfitting: How to modify the Neural Network Architecture?

7. Modify the weights computation, modify the class weights, add drop-out layers
   - If strong overfitting.. Restart the game from the data preprocessing/design the architecture

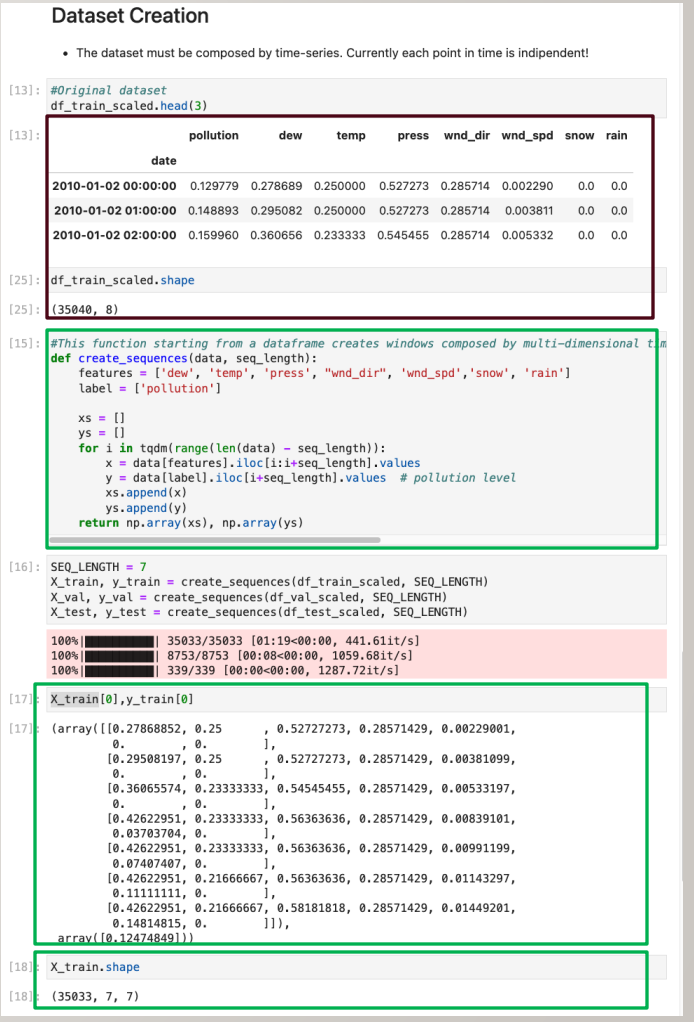
RECURRENT NEURAL NETWORK: A SUMMARY OF HYPERPARAMETERS AND BEST PRACTICES

Hyperparameter	Best Practices
#Architecture	Mono-directional or Bi-directional RNN, LSTM based on what we can learn from the sequence order backword and how long the memory should be
# Layers	Start with 1-2 RNN layers, increase if needed
# Hidden Size	Use powers of 2 (64, 128, 256), avoid too many
Activation	Typically tanh for hidden Typically,ly, Softmax/Sigmoid/linear for output
Weight Initialization	PyTorch automatically initialize the weights with different methods based on the activation function in the layer. Manual initialization is possible also for the hidden/cell states
Time-series with Different Length	If different time-series have different length we may have to add padding/cutting the time-series
Batch Size	32-256, tune experimentally
Batch Shuffling	Consider shuffling if different sequences are not related to each other
Loss Function	Choose based on the task
Optimizer	Adam (Default), SGD+Momentum for generalization
Learning Rate	Start with 0.001, use LR scheduler.
Epochs & Early Stopping	Monitor validation loss, stop if overfitting. The number of epochs can drastically change based on the optimizer and Learning Rate!
Regularization	Dropout (0.2-0.5) (Notice can be inserted as separate layers or directly in **the RNN nodes as well**) + L2 (0.01)
#Architecture <u>After</u> RNN	After the RNN consider if it is required to add additional linear/non-liner layers or architectures based on task and performance

LOADING THE DATA: FROM RAW DATA TO TENSOR

After preparing the data.. Machine Learning **starts** from loading the data in pyTorch

- Time-series data data must be **3D matrix**
  - Based on the initial dataset preprocessing must transform
- **2D problem** each line is a point in time x(t) and each element is a feature
  - I.e., 35040 x(t) with 7 features + 1 label each
- **Create_sequence** transforms the matrix in 3D: each matrix represent a time-series with 7 points
  - I.e., 35033 matrices each with 7 x(t) described by 7 features each point
    - **We remove the label**



Dataset Creation

- The dataset must be composed by time-series. Currently each point in time is indipendent!

```
[13]: #Original dataset
      df_train_scaled.head(3)
```

[13]:

date	pollution	dew	temp	press	wnd_dir	wnd_spd	snow	rain
2010-01-02 00:00:00	0.129779	0.278689	0.250000	0.527273	0.285714	0.002290	0.0	0.0
2010-01-02 01:00:00	0.148893	0.295082	0.250000	0.527273	0.285714	0.003811	0.0	0.0
2010-01-02 02:00:00	0.159960	0.360656	0.233333	0.545455	0.285714	0.005332	0.0	0.0

```
[25]: df_train_scaled.shape
```

```
[25]: (35040, 8)
```

```
[15]: #This function starting from a dataframe creates windows composed by multi-dimensional tim
      def create_sequences(data, seq_length):
          features = ['dew', 'temp', 'press', "wnd_dir", 'wnd_spd','snow', 'rain']
          label = ['pollution']

          xs = []
          ys = []
          for i in tqdm(range(len(data) - seq_length)):
              x = data[features].iloc[i:i+seq_length].values
              y = data[label].iloc[i+seq_length].values  # pollution level
              xs.append(x)
              ys.append(y)
          return np.array(xs), np.array(ys)
```

```
[16]: SEQ_LENGTH = 7
      X_train, y_train = create_sequences(df_train_scaled, SEQ_LENGTH)
      X_val, y_val = create_sequences(df_val_scaled, SEQ_LENGTH)
      X_test, y_test = create_sequences(df_test_scaled, SEQ_LENGTH)

      100%|██████████| 35033/35033 [01:19<00:00, 441.61it/s]
      100%|██████████| 8753/8753 [00:08<00:00, 1059.68it/s]
      100%|██████████| 339/339 [00:00<00:00, 1287.72it/s]
```

```
[17]: X_train[0],y_train[0]
```

```
[17]: (array([[0.27868852, 0.25      , 0.52727273, 0.28571429, 0.00229001,
                0.        , 0.        ],
               [0.29508197, 0.25      , 0.52727273, 0.28571429, 0.00381099,
                0.        , 0.        ],
               [0.36065574, 0.23333333, 0.54545455, 0.28571429, 0.00533197,
                0.        , 0.        ],
               [0.42622951, 0.23333333, 0.56363636, 0.28571429, 0.00839101,
                0.03703704, 0.        ],
               [0.42622951, 0.23333333, 0.56363636, 0.28571429, 0.00991199,
                0.07407407, 0.        ],
               [0.42622951, 0.21666667, 0.56363636, 0.28571429, 0.01143297,
                0.11111111, 0.        ],
               [0.42622951, 0.21666667, 0.58181818, 0.28571429, 0.01449201,
                0.14814815, 0.        ]]),
        array([0.12474849]))
```

```
[18]: X_train.shape
```

```
[18]: (35033, 7, 7)
```

LOADING THE DATA: FROM RAW DATA TO TENSOR

What if time-series have different length?

- We divide the data in mini-batches
  - Working with matrices, **inside the same mini-batch** data must have the **same number of x(t)** but **different mini-batches CAN** use different number of x(t)! ☺
    - **Q:** In FFNN could we use input of different size?
    - **A**: No, this is one of the benefits of working with indented matrices
  - What if inside the same mini-batch data have different number of x(t) ☹
    1. We add padding to the time-series shorter than the longest one
    2. We cut the longest time-series
  1. We can add the padding by creating a **class** to implement a custom *TensorDataset()*
     - The *__init__* function defines how data is passed to create the dataset
     - The *__getitem__* returns one element of the series
     - The ***collate_fn*** adds the padding! Better than cutting (we **do not** remove useful data from a time-series)
       1. Identify the longest time series
       2. Add padding to the time series shorter w.r.t. the longest using the *pad_sequence* function!
  2. Specify in the *DataLoader* to use the ***collate_fn*** function. It will use it <u>if needed</u>
     - **ONLY** if time-series in the **same mini-batch** have a different number of samples!

```python
# Custom Dataset
class TimeSeriesDataset(Dataset):
    def __init__(self, time_series_list, labels_list):
        self.time_series_list = time_series_list  # List of sequences
        self.labels_list = labels_list            # List of labels

    def __len__(self):
        return len(self.time_series_list)

    def __getitem__(self, idx):
        sequence = torch.tensor(self.time_series_list[idx], dtype=torch.float32)
        label = torch.tensor(self.labels_list[idx], dtype=torch.long)
        return sequence, label
```

```python
def collate_fn(batch):
    sequences, labels = zip(*batch) # Unpack batch
    lengths = torch.tensor([len(seq) for seq in sequences]) # Sequence lengths

    # Sort sequences by length (descending) to optimize RNN
    sorted_indices = lengths.argsort(descending=True)
    sequences = [sequences[i] for i in sorted_indices]
    labels = torch.tensor([labels[i] for i in sorted_indices])
    lengths = lengths[sorted_indices]  # <-- Sort lengths too

    # Pad sequences
    padded_sequences = pad_sequence(sequences, batch_first=True, padding_value=0)

    return padded_sequences, lengths, labels
```

```python
# Create DataLoader
train_dataset = TimeSeriesDataset(X_train_to_pad, y_train)
val_dataset = TimeSeriesDataset(X_val, y_val)

train_loader = DataLoader(train_dataset, batch_size=9,collate_fn=collate_fn, shuffle=False)
val_loader = DataLoader(val_dataset, batch_size=11, collate_fn=collate_fn, shuffle=False)
```

CRATE A RECURRENT NEURAL NETWORK ARCHITECTURE

PyTorch creates NN defining it with a class

1. Create the class to describe the NN architecture as for the FFNN
2. The ***__init__*** function can have multiple parameters
   - E.g., number of input neurons, number of layers, number of neurons per layer, activation function, etc..
   - The ***super*** constructor specify that it is a NN
   - **The RNN layer** gets a number of input = to the number of input neurons, the hidden size (i.e., the number of features in the hidden state), the number of layers
     - I.e., The features describe how to represent the data in the hidden state h
   - **The output layer here** has the input size as the size of the hidden layer size
     - The neurons in the output layer depends on: the task regression/classifier, number of classes, Loss function
     - We may add multiple layers (e.g., drop-out, non-linear) between the RNN layer(s) and the output layer
3. The ***forward*** function defines how data pass trough the NN

```python
class RNNPollutionPredictor(nn.Module):
    """
    Args:
      - input_size – The number of expected features in the input x
      - hidden_size – The number of features in the hidden state h
      - output_size – The number of expected outputs
    """
    def __init__(self, input_size, hidden_layer_size, output_size):
        super(RNNPollutionPredictor, self).__init__()
        self.hidden_layer_size = hidden_layer_size
        #batch first is used to specify that the matrix in input has dimension:
        #(batch_size, sequence_length, input_size)
        self.rnn = nn.RNN(input_size, hidden_layer_size, batch_first=True)
        self.linear = nn.Linear(hidden_layer_size, output_size)

    """
    Args:
        - input_seq: The input sequence tensor of shape
         (batch_size, sequence_length, input_size).
     Output of rnn:
        - **rnn_out** contains the output for all time steps.
          It has shape (batch_size, sequence_length, hidden_size).
        - **h** contains the hidden states for all layers at the last time step.
           It has shape (num_layers, batch_size, hidden_size).
        - if different time-series have different sequence_length, rnn_out can generate errors
        - with h we get batch_size instead of sequence_length
          ==> no problem with different sequence_length

        - We do not return them in the tracking loop to
            - avoid problems with the gradient vanish/exploding
            - use the windows independently (if possible) and shuffle the data
        Returns:
         - predictions: The predicted pollution levels for the input sequence.
         Use h from last hidden node helps with time-series having different lengths
    """
    def forward(self, input_seq):
        rnn_out, h = self.rnn(input_seq)
        predictions = self.linear(h[-1])
        return predictions
```

CRATE A RECURRENT NEURAL NETWORK ARCHITECTURE: HOW TO CONSIDER THE PADDING?

- RNN architecture to **handle variable-length**
  - RNN uses the last **hidden state** for classification.
- Inputs may have different shape 2D instead of 3D
  - x = x.unsqueeze(-1): add the missing dimension

```python
# Define RNN Model
class RNNClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes):
        super(RNNClassifier, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True, nonlinearity="tanh")
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x, lengths):
        x = x.unsqueeze(-1)  # Add extra dimension
        # Pack padded sequence for efficiency
        packed_x = pack_padded_sequence(x, lengths.cpu(), batch_first=True, enforce_sorted=True)

        out, hidden = self.rnn(packed_x)  # Get last hidden state
        return self.fc(hidden[-1])  # Use last layer's hidden state for classification because we are working with padding
```

- Efficiently processes padded sequences using *pack_padded_sequence()* function
  - Do not compute the RNN on the padding. Avoid bias introduced by the padding
- Maps the **hidden state** to **class scores** with the last linear layer
  - Use the Hidden state since sequences are padded, the last time-step varies across samples
  - The hidden state ensures we get a fixed-size representation.
  - This avoids relying on padded parts of the sequence during classification.

CRATE A RECURRENT NEURAL NETWORK ARCHITECTURE: DIFFERENT ARCHITECTURE

- To use a **bi-directional** RNN
  1. Specify the RNN architecture
  2. In the liner layer specify the correct number of neurons
     - 2 directions ➔ 2x hidden size
  3. In the forward loop use both hidden states directions for the liner layer
- To use a **LSTM**
  1. Specify the LSTM architecture
  2. In the forward loop remember that LSTM return both the **hidden** state and the **cell** state for long-term memory

```python
# Define Bi-directional RNN Model
class BiRNNClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes):
        super(BiRNNClassifier, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True, bidirectional=True)
        self.fc = nn.Linear(hidden_size * 2, num_classes)  # Multiply by 2 for bidirectional

    def forward(self, x, lengths):
        x = x.unsqueeze(-1)  # Add extra dimension
        # Pack padded sequence for efficiency
        packed_x = pack_padded_sequence(x, lengths.cpu(), batch_first=True, enforce_sorted=True)

        packed_out, hidden = self.rnn(packed_x)  # Get last hidden state
        out, _ = pad_packed_sequence(packed_out, batch_first=True)

        # Concatenate the final forward and backward hidden states
        hidden_cat = torch.cat((hidden[-2], hidden[-1]), dim=1)

        return self.fc(hidden_cat)  # Use concatenated hidden states for classification
```

```python
Define LSTM Model
ass LSTMClassifier(nn.Module):
  def __init__(self, input_size, hidden_size, num_layers, num_classes):
      super(LSTMClassifier, self).__init__()
      self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
      self.fc = nn.Linear(hidden_size, num_classes)

  def forward(self, x, lengths):
      x = x.unsqueeze(-1)  # Add extra dimension
      # Pack padded sequence for efficiency
      packed_x = pack_padded_sequence(x, lengths.cpu(), batch_first=True, enforce_sorted=True)

      packed_out, (hidden, cell) = self.lstm(packed_x)  # Get last hidden state
      out, _ = pad_packed_sequence(packed_out, batch_first=True)
      # Use last layer's hidden state for classification
      #because here there is padding
      return self.fc(hidden[-1])
```

DEMO

Try with PyTorch!